

Efficient Quantum Sobel Edge Detection

A Thesis submitted to the faculty of

San Francisco State University

In partial satisfaction of the

requirements for

the Degree

Masters of Science

in

Computer Science

by

Christian M. Pestano

San Francisco, California

May 2025

Copyright by
Christian M. Pestano
2025

Certification of Approval

I certify that I have read Efficient Quantum Sobel Edge Detection by Christian M. Pestano and that in my opinion this work meets the criteria for approving a thesis submitted in partial fulfillment of the requirement for the degree Masters of Science at San Francisco State University.

E. Wes Bethel, Ph.D
Associate Professor
Thesis Committee Chair

Daniel Huang, Ph.D
Assistant Professor

Talita Perciano, Ph.D
Research Scientist

Abstract

This paper augments the edge-detecting Sobel Filter with quantum calculations. This implementation replaces the classical stencil operation, which calculates the horizontal and vertical gradients at each pixel through a dot product of edge detection kernels and its neighbors, with a quantum operation that allows each pixel address to gain access to all neighboring values simultaneously. This was done by encoding shifted versions of the original image into a quantum circuit using QCrank and performing the weighted sum calculation within the quantum circuit using the EHands protocol, both developed by LBNL. This thesis also developed a novel reversible version of the EHands parity flip gate. As an additional optimization, this Sobel implementation added functionality to break the input image into tiles on which smaller quantum circuits would act, with their results reassembled into a final image. The experiments of this paper verify the effectiveness of the EHands blocks and gradient calculation circuits, and compare simulation runtime and circuit structure for various combinations of circuit variants and quantum calculations needed for the Sobel Filter. These experiments found the optimal circuit variant for calculating Sobel stencil operations, that GPU nodes outperform CPU nodes at all images sizes, and that divide-and-conquer tiling is superior to processing the input image in its entirety for circuit simulations with larger image sizes. Future work aims to extend the study of this implementation on real quantum hardware and introduce further optimizations to the implementation.

Acknowledgments

Thank you to Dr. Bethel for his guidance and support throughout not only the development of this thesis, but for much of my journey in SFSU. Thank you to the Lawrence Berkeley National Labs AQuA-DATA Team, especially Dr. Perciano and Dr. Balewski for their advice and guidance on utilizing their algorithms and best practices for creating reliable quantum circuits. Thank you again to Dr. Perciano, as well as Dr. Huang, for taking time out of their schedule to serve as members of my thesis committee. Thank you to the Metro Program's Data Team, who encouraged me to continue pursuing higher education. Last but not least, thank you to my parents for their constant support and sacrifice.

Table of Contents

Table of Contents	vi
List of Tables	viii
List of Figures	ix
1 Background and Literature Review	5
1.1 Image Processing and Potential for Quantum Acceleration	5
1.2 Quantum Computing Basics	8
1.3 Quantum Image Processing	13
1.4 QSobel	16
1.5 LBNL Quantum Algorithms	18
1.6 Qiskit	24
2 Implementation	26
2.1 Overall Sobel Circuit Architecture	27
2.2 EHands Arithmetic	44
3 Evaluation and Results	52
3.1 Computational platform and Software Environment	53
3.2 Methodology	54
3.3 Experiment 1: Verification of Correctness of Quantum Arithmetic Computations	62
3.4 Experiment 2: Comparison of Extended and Compact Circuit Variants . . .	65
3.5 Experiment 3: CPU vs. GPU Runtime Comparison	68
3.6 Experiment 4: Comparison between Tiling and Increasing Circuit Size	70
3.7 Experiment 5: Comparison of Tiling Sizes for a Fixed Size Image	72
3.8 Findings and Discussion	75
3.9 Potential Future work	79
4 Conclusion	82
Bibliography	84

Appendices	88
.1 Appendix A: Verification of Effectiveness	88

List of Tables

2.1	The tradeoffs of moving the Sobel calculation back to classical computing at various points in the process.	29
3.1	Summary of key values in the 20 output analysis charts created during Experiment 1. The two parity circuits were tested via a sequence of weight summation blocks. All produce intended results. All charts used to build this table can be found in the Appendix.	64
3.2	A comparison of circuit shape and number of 1 and 2 qubit gates between the compact and extended circuit variants for various image sizes. The circuits have similar shapes and gate counts, but any difference in circuit width is impactful at larger values.	67
3.3	A comparison of circuit shape and number of 1 and 2 qubit gates between the compact and extended circuit variants for various image sizes.	71
3.4	Shows the relationship between tile length and how many tiles are needed to fill out one dimension of the 256×256 image if no overlapping is used. The third column calculates the amount of overlapping addresses are allotted to each tile in one dimension of the image.	73
3.5	Summary of the amount of tiles used when each tile is a given length, as well as the overall runtime and the average runtime per tile length. Tile length 16 has the smallest runtime per tile, but has a higher total serial runtime compared to tile length 16 because more extra tiles are required in each dimension of the image	75

List of Figures

1.1	The filter windows a) G_x used to detect the vertical edges and b) G_y used to detect horizontal edges	7
1.2	A diagram of a Bloch Sphere [13]	10
1.3	An example of how a right shift operation produces a new image where each address has the value of its left neighbor.	17
1.4	The initial versions of the EHands (a) encoding and decoding (b) multiplication (c) weighted sum (d) negation, and (e) parity flip sub-circuits [3]	22
2.1	A flowchart of the overall implementation of just the G_x and G_x^2 calculations. A G_y and G_y^2 calculation flowchart would similar, and the full $\sqrt{G_x^2 + G_y^2}$ was too large of a circuit to simulate on classical hardware. Additionally real QPUs were unavailable during testing, so the full Sobel gradient calculation needed to be done piecemeal using smaller quantum circuits	27
2.2	How each neighbor of a pixel in the G_x and G_y kernel is mapped to one of the 6 data qubits in each of the extended Sobel's stencil circuits.	31
2.3	The circuit diagram of the extended Sobel convolution portion of the quantum circuit. The QCrank data encoding sub-circuit located to the left of the circuit and address qubits located below the circuit were left out for space. Note the negations placed on last 3 blocks, which represent subtraction between the positive and negative portions of the filter kernel. Note that the barriers included here are for readability and removed during the execution of the circuit	32
2.4	The circuit diagram of the compact Sobel convolution portion of the quantum circuit. As with the extended circuit diagram, the QCrank data encoding located to the left of the circuit and address qubits located below the circuit were omitted for brevity. Note the negations placed on middle 3 blocks, which represent subtraction between the positive and negative portions of the filter kernel. Note that the barriers included here are for readability and removed during the execution of the circuit	34
2.5	How each neighbor of a pixel in the G_x and G_y kernel is mapped to one of the 6 data qubits in each of the compact Sobel's convolution circuits.	35

2.6	The implementation of a G_x^2 or G_y^2 calculation circuit, which uses 2 G_x or G_y circuits stacked on top of each other, with their results multiplied together. Note that the barriers included here are for readability and removed during the execution of the circuit	39
2.7	Comparison of the expected Sobel result, which was computed classically, with the combined results of 4 tiles of quantum-augmented Sobel made up of the quadrants of the original image. The images themselves are grayscale, with the red lines added to better show the area of each tile. The fourth image highlights the tile borders, which are not calculated correctly because not all of their neighbors used in their Sobel stencil operations are valid.	40
2.8	Comparison of the expected Sobel result, which was computed classically, with the combined results of 9 tiles of quantum-augmented Sobel made up of the quadrants sized tiles of the original image. The images themselves are grayscale, the red, blue, and green shaded boxes representing the tiles used for the bottom row of the image	41
2.9	Circuit comparisons between the original EHands parity flip block and the reversible EHands parity flip block developed for this thesis. The mid-circuit measurement is replaced by a CZ gate.	48
3.1	This chart displays which combinations of circuit variants and image sizes fit into the Perlmutter CPU and GPU nodes. Combinations that produce a circuit width of 34 are the only ones that can fit in a CPU node but not in the GPU memory of the GPU nodes.	58
3.2	Two graphs used to verify the correctness of computations of the extended quantum-augmented Sobel circuit when calculating G_x by testing it on a randomly generated 32×32 image and comparing it to scaled down fully classically calculated values. Both produce expected results, suggesting the circuit is working as intended.	63
3.3	A comparison of runtimes between the compact circuit implementation and the extended implementation when calculating G_x at various image sizes. Note log scaling on the y-axis. The compact circuit is faster at all image sizes	66
3.4	A comparison of runtimes of the G_x calculation when ran on a CPU node vs a GPU node for various image sizes.	68
3.5	A comparison of the original 128×128 image, the fully classical Sobel comparison, and the Quantum-Augmented Sobel output.	69
3.6	A comparison of runtimes between no tiling, tiling with quadrant sized tiles, and the theoretical runtimes if the tiles were calculated in parallel. The serial tiling execution only starts to outperform the implementation that uses the full image at the 256×256 image.	71

3.7	A graph depicting how runtime of the G_x computation on a 256×256 image changes with length of tiles. Tile length 16, which needed 3 extra tiles per image dimension instead of 1, produces a runtime that does not follow the pattern of the other tile length. While these other values produce a straight line on this figure, note that logarithmic scaling of y-axis, which instead suggests an exponential scaling of runtime.	74
.1	Accuracy Verification for EHands Encoding with 100 randomly generated samples	88
.2	Verification of Effectiveness for Ehands Multiplication with 100 randomly generated samples	88
.3	Verification of Effectiveness for EHands Weighted Sum with 100 randomly generated samples	89
.4	Verification of Effectiveness for the new parity flip block using a sequence of EHands weighted summations with 100 randomly generated samples	89
.5	Verification of Effectiveness for the old parity flip block using a sequence of EHands weighted summations with 100 randomly generated samples	90
.6	Verification of Effectiveness of the G_x compact circuit with a 32×32 image . . .	90
.7	Verification of Effectiveness of the G_y compact circuit with a 32×32 image . . .	91
.8	Verification of Effectiveness of the G_x^2 compact circuit with a 32×32 image . . .	91
.9	Verification of Effectiveness of the G_y^2 compact circuit with a 32×32 image . . .	92

Introduction

Many image processing techniques involve a large number of repetitive calculations, due to the need to perform operations for each pixel coordinate. The Sobel Operator[16], which is used to detect horizontal and vertical edges in an image, is one such technique. The gradient G_x detects vertical edges by calculating the difference in values along the x-axis of its kernel window, while G_y calculates the horizontal edges by calculating the difference in values along the y-axis of its kernel window. These kernel windows are applied to each pixel, with a dot product of the each gradient's respective kernel windows applied to the neighboring values of each pixel. By adding the squares of the two gradients together and applying a square root operation on their sum, information regarding edge detection for both axes can be stored in one value as the gradient magnitude. The dot product portion of the Sobel Filter, the stenciling or convolution operation, is normally accelerated with parallelization via GPUs, but this process for each pixel can be handled simultaneously for all image addresses by leveraging quantum mechanics. Zhang et al. proposed QSobel[21], a method to encode multiple shifted versions of the original input image to gain information from neighboring values simultaneously. However, QSobel does not provide a means of scaling or adding values within the quantum circuits. Additionally, it did not provide enough detail to replicate shifting images and storing them within quantum circuits.

The methods in this paper utilize QCrank[4], a method for efficiently encoding multiple data channels for images in a quantum circuit, and EHands[4], which allows for basic arith-

metric between values encoded on quantum circuits, to expand on the work introduced by QSobel. After this implementation creates shifted images via classical computing methods, it then loads the images into several data channels using QCrank, and computes the gradients within the quantum circuit using summation and multiplication sub-circuits defined by EHands. With this method, the implementations developed in this thesis use quantum circuits to compute calculations for G_x , G_y , G_x^2 , G_y^2 , or $G_x^2 + G_y^2$. In addition to the implementation of these calculations with quantum circuits, this thesis contributes to the EHands protocol the design for a novel reversible version of the EHands parity flip block, which until this paper was the only irreversible EHands operation. By allowing full reversibility for the EHands protocol, the potential applications have expanded to purposes that previously needed fully reversible circuits. This project also implemented the option to divide the picture into smaller tiles to spread the computation into smaller quantum circuits, improving simulation runtime and potentially also decreasing quantum error impact on real QPUs.

Because EHands does not have a circuit for square root operations and no valid method for shifting and storing images purely with quantum circuits has been found, the rest of the Sobel operations must be calculated classically. However, because real QPUs were unavailable during the development of this thesis, only some of these circuits were realized via simulation due to memory and runtime limitations when simulating on classical hardware. With this in mind, the impact of transitioning back to classical computations at different points in the Sobel Filter implementation were conducted to find the optimal point to switch back to classical computation when using simulations.

This thesis conducted five experiments for this quantum-augmented Sobel implementation. The first verifies the effectiveness of the EHands arithmetic blocks, including the novel reversible design for the EHands parity flip block on ideal simulators using IBM’s Qiskit. It also tested that the stenciling operations and squared stenciling operations that fit into classical memory were producing the correct results by comparing their outputs with fully classical Sobel computations. The second experiment compares the runtimes and quantum circuit characteristics of circuit variants for calculating the Sobel stenciling: a larger, yet more straightforward implementation, and a compact variant. The third experiment verifies that the full quantum-augmented Sobel process produced results comparable to the original Sobel Operator, and was conducted on both CPU and GPU nodes at Perlmutter to also compare the runtime between the two types of nodes. The final two experiments examined the effectiveness of the tiling functionality in simulation runtime. The first compares the runtime of the Sobel G_x gradient calculation for circuits using the entire image with the total runtime of circuits using tiles of the original input. The second examines how changing the size and count of tiles affected simulation runtime.

These experiments verify that all circuits perform as intended, with the compact circuit variant outperforming the extended circuit variant with regard to simulation runtime. It was additionally found that for simulations, squaring the gradient outputs in the quantum realm had too large of an impact on simulation runtime, and GPU nodes outperformed CPU nodes at all tested image sizes. Lastly, it was found that the tiling only has a benefit at larger image sizes due to extra tiles needed to produce tile overlap to cover invalid tile

border regions and decreasing the length of the tiles provided an exponential improvement in runtime. With these positive results, further improvements to this quantum-augmented approach can be implemented, and testing on real quantum hardware with larger images and more calculations can be conducted.

Chapter 1

Background and Literature Review

1.1 Image Processing and Potential for Quantum Acceleration

Image processing is a long-standing area of focus in Computer Science. With an ever increasing focus on computers in both daily life and professional applications, efficient and effective image creation and processing has become more important [10]. Image processing plays a role in a variety of fields, such as preprocessing images to increase the accuracy of machine learning algorithms. This is particularly useful for important tasks such as the identification of diseases in the medical field [20].

Edge Detection and Sobel Filtering

Edge detection is a category of image processing that can be used to clean images to make them easier for AI or humans to notice key elements in them. As its name suggests, these types of techniques focus on finding points of high gradients in images [16]. They can be used in conjunction with a thresholding operation to suppress lower gradient signals, creating solid distinct areas in an image. The Sobel Filter, or Sobel Operator [16], is one popular edge detection algorithm. This filtering operation has two filter windows shown in Fig. 1.1: a vertical edge detection window used to calculate a) G_x and a vertical edge detection window used to calculate b) G_y . A dot product, between the filter windows and the values of the address they correspond to is used to calculate each gradient. For the purposes of this paper, calculations G_x and G_y will be referred to as stencil operations or convolutions, for the sake of conciseness.

The signal values G_x and G_y represent a change in the values along the respective directions of each window. Squaring the resulting values into values Gx^2 and Gy^2 ensures positive signals for both gradients, and adding the values together creates a single value that contains information on both filter directions. Taking the square root of this final value produces a final Sobel gradient magnitude value $G = \sqrt{Gx^2 + Gy^2}$ that is on the same scale as the original pixel values. It should be noted that the choice of which side for each kernel that is negative does not matter due to the squaring step in the calculation, so they can be assigned arbitrarily and the exact windows can differ between papers. The experiments carried out

-1	0	1	-1	-2	-1
-2	0	2	0	0	0
-1	0	1	1	2	1
a) G_x			b) G_y		

Figure 1.1: The filter windows a) G_x used to detect the vertical edges and b) G_y used to detect horizontal edges

in this paper will follow the order utilized in the QSobel[21] paper for simplicity.

Accelerated Processing and Image Tiling

Given the repetitive, yet often computationally expensive nature of many image processing techniques such as the Sobel filter, a popular area of development for image processing is accelerating the calculations. This can be achieved in a variety of ways, such as parallelizing image processing computations with GPUs[6]. By dividing an image into smaller tiles and distributing them across threads in a GPU, smaller versions of the work can be done

simultaneously, speeding up the computation process. This idea of tiling an image is used in this paper, but for different reasons related to the quantum version's implementation, as discussed in Sec. 2.

1.2 Quantum Computing Basics

Quantum computers work in a fundamentally different way compared to more commonly used classical computers. Similarly to bits in classical computers that exist in the discrete states of 0 and 1, quantum bits, or qubits, can exist in the basis states $|0\rangle$ and $|1\rangle$ [13]. However, unlike bits, qubits can also exist in a state of superposition between $|0\rangle$ and $|1\rangle$, which corresponds to a linear combination of the two basis states, denoted by the equation

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle. \quad (1.1)$$

This equation is also the simplest form of a state vector, which is a complex unit vector describing the state space of a qubit or system of qubits. To represent a system of entangled qubits, the state vector format in Eq. 1.1 can be expanded to represent the probability of each combination of basis states. When a qubit is measured, its superposition state collapses into $|0\rangle$ and $|1\rangle$, with the probability of each represented by the square of the probability amplitudes α and β . These amplitudes are complex numbers that represent the probability amplitudes of their respective basis states, which additionally apply the constraint $|\alpha|^2 + |\beta|^2 = 1$. With this additional constraint, a qubit's basis state can be

represented by the following formula:

$$|\psi\rangle = \cos\frac{\theta}{2} |0\rangle + e^{i\phi} \sin\frac{\theta}{2} |1\rangle. \quad (1.2)$$

Using this formula, the θ and ϕ terms of the quantum state ψ may be plotted on the Bloch sphere similar to that in Fig. 1.2, where the poles on the Z-axis represent $|0\rangle$ and $|1\rangle$. The distance to each pole corresponds to the probability amplitude and is controlled by the angle θ , with ϕ controlling the phase, or rotation along the x-y plane of the sphere. Although intuitively rotations along the angle ϕ do not result in different probability amplitudes, certain operations on the Bloch sphere are affected by these rotations. Quantum gates can be applied to qubits to influence their states, with each gate's impact on the quantum state of one or more qubits defined by a unitary matrix. Additionally, physical objects or phenomena used to represent qubits, such as the spin of trapped ions, can become entangled, where the state of one qubit has an impact on the state of its entangled qubits [5]. A Bloch sphere has limited uses, as it can only represent the state of one qubit.

A density matrix is an alternative to the state vector approach, which describes a quantum system by the probabilities that it is in known pure states, where each pure state is a fully known quantum state [13][15]. Although this thesis does not utilize the density matrix approach for quantum state representation, the approach is necessary to describe when a system's state is no longer fully known. Noise can cause incomplete knowledge of a system in real quantum systems, but for this thesis, which conducts its experiments on simulators, mixed states will occur when an irreversible operation, like a measurement, is applied to a

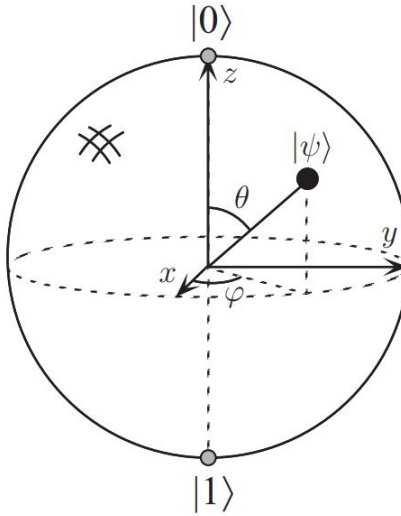


Figure 1.2: A diagram of a Bloch Sphere [13]

circuit.

Quantum Circuits and Gates

A quantum circuit is a system of qubits, and quantum logic gates apply operations to a qubit or a group of qubits within the circuit to change the system's quantum state [15]. Each quantum gate has a unitary matrix that represents the operation that the gate applies to a qubit to change its state. For example, a simple X, or NOT gate, has the following matrix:

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad (1.3)$$

When applied to a qubit in state $|0\rangle$, the qubit is transformed into state $|1\rangle$.

$$X|0\rangle = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} = |1\rangle \quad (1.4)$$

Unitarity is the only constraint for quantum gates [13], so there is a wide variety of gates that can be applied to quantum circuits. This section will only discuss gates used by the circuits in this thesis, with information regarding each taken from the Qiskit documentation¹. The X gate, or NOT gate, as previously mentioned, is the quantum equivalent of a classical bit flip gate, creating a π radian rotation about the X axis of a qubit. The Ry is a single-qubit gate that creates a programmable rotation about the Y axis. Similarly, the Rz gate creates a programmable rotation about the Z axis. The Hadamard gate, or the H gate, provides a π rotation about the X+Z axis. An H gate brings a qubit in the state $|0\rangle$ or $|1\rangle$ to a state of equal superposition [15]. The CX, or CNOT gate, is 2-qubit gates that conditionally apply an X gate to a target qubit if its control qubit is in state $|1\rangle$. Similarly, the CZ gate is a 2-qubit gate that applies a Z gate to its target qubit if its control qubit is in state $|1\rangle$.

Circuit designers often replace more complex gates with sub-circuits of simpler gates to be more compatible with a wider variety of Quantum Processing Units (QPUs). For example, this thesis will discuss Uniformly Controlled Rotation (UCR) gates in Sec. 1.5. UCR gates are multi-parameter, multi-qubit gates acting on several control qubits and 1 target qubit[4]. The LBNL team replaced UCRy gates, which are UCR gates that apply Ry rotations, with equivalent sub-circuits of Ry and CNOT gates [1].

¹Qiskit documentation online at <https://docs.quantum.ibm.com/>, May 2025

Although the quantum measurement operation is not a gate operation because it is not reversible, the experiments in this thesis that examine quantum circuit architecture group measurement operations with the 1-qubit gate operations because they are still single-qubit operations applied to a circuit.

Advantages and Disadvantages of Quantum Systems

Because qubits in a quantum circuit exist in a state of superposition until measured, they are not deterministic yet are still predictable in an ideal environment, allowing them to perform multiple calculations simultaneously by repeatedly executing shots of a circuit [5]. Quantum computers can explore a larger space of values compared to classical computers, which is used for things like the Grover Search algorithm, which is more efficient than its classical counterparts by leveraging quantum superposition and quantum parallelism to search its entire state space simultaneously [15][7]. As the shot count increases, the accuracy of the results approaches the expected probabilities of the overall circuit's state vector. The nondeterministic nature of quantum superposition also allows for increased space efficiency. In a classical computer, an n -bit register representing an unsigned integer can encode a single value from 0 to $2^n - 1$, while a quantum register of n qubits in superposition can simultaneously represent all values from that same range. By manipulating the entanglement and states of individual qubits, it is possible to adjust the state vector of a system to increase the probabilities of the intended results.

Quantum unitary operations, such as those created with many quantum gates, are also reversible and invertible. This allows for the inference of a circuit's original value based on its results because the output of a unitary operation can only have one input. This is important for quantum operations, as reversibility means that no information which might be needed for operations later in a quantum circuit is destroyed during the operation of the circuit. Because measurement and reset operations collapse the state of the measured qubits, these actions are irreversible, producing a mixed state from the previous pure state which was perfectly predictable[15].

A major disadvantage of quantum systems is that the class of Noisy Intermediate-Scale Quantum (NISQ) machines, as their name suggests, are still noticeably affected by noise that can adversely impact their functions [5]. Although there are constant advances in error correction and minimizing the physical faults in QPUs that produce incorrect results, large-scale quantum operations cannot be performed on QPUs because of compounding errors. With NISQ computers, the combination of relatively small numbers of qubits combined with high error rates significantly limits the size of circuits that are possible on present-day hardware.

1.3 Quantum Image Processing

A popular topic in quantum computing is how to leverage its characteristics in the field of image processing, known as Quantum Image Processing (QIMP), or quantum image rep-

resentation (QIR). The predictable repetitive nature of many image processing techniques that let them work well with GPU acceleration can also fit well with quantum circuits.

Popular QIMP Encoding Methods

Although there are many ways to encode information into a quantum system and sometimes different terms used to describe each encoding method [15][14], for QIMP three of the most common methods are basis encoding, amplitude encoding and angle encoding[18][4].

Basis encoding is the simplest form of encoding, where n -bit strings are encoded into n -qubit systems, with the basis state with highest probability amplitude the output of the system. Although reliable, this method does not make full use of the space efficiency that quantum computing offers. This method is effective when encoding discrete values, and, as such, is popular for encoding address information in QIMP. Amplitude encoding involves storing normalized real or complex data vectors in the amplitudes in a system's state vector. Angle encoding is similar to amplitude encoding in that data is retrieved by examining the probability amplitude of a state vector, but involves encoding information through an angle-rotation gate applied to a data qubit. This method is very popular in QIMP and is heavily utilized in the code for this thesis. The information is decoded or retrieved by estimating the angle value based on the probability distribution that results from the execution of the circuit. A disadvantage of this method is that because information is dependent on probability amplitudes of the system's state vector, certain mathematical operations, such

as methods that cannot be mapped to the probability amplitudes whose squared values must sum to 1, are incompatible with this method. Additionally, the values stored in the probability amplitudes can be complex numbers, but the probability values recovered during measurement are real values. As a result, any information stored in the imaginary portion of a statevector's probability amplitudes will not be reflected in the probability distributions. However, this is not a problem with QIMP, as the image data values are strictly real numbers, so no information is lost.

FRQI

FRQI[12] is considered a foundational encoding method for quantum image processing, with many other methods developed from it[18]. It utilizes angle encoding to store pixel values. More specifically, this model is defined by Eq. 1.5, where θ_i is from the range 0 to $\frac{\pi}{2}$ and represents the pixel data for each element i . $(\cos\theta_i |0\rangle + \sin\theta_i |1\rangle)$ encodes the pixel data, while $|i\rangle$ is the portion of the basis states corresponding to that data's pixel coordinates.

$$|I(\theta)\rangle = \frac{1}{2^n} \sum_{i=0}^{2^{2n}-1} (\cos\theta_i |0\rangle + \sin\theta_i |1\rangle) \otimes |i\rangle \quad (1.5)$$

This formula is commonly represented in a quantum circuit with a UCR gate or some other multi-controlled angle gate for each pixel value controlled by qubits representing the x and y addresses. Each multi-controlled angle gate, or equivalent sub-circuit, only applies its angle rotation when all the address qubits that represent its address are in state $|1\rangle$. For a $2^n \times 2^n$ grayscale image, the x and y address space need n address qubits each, for

a total circuit width of $2n + 1$. Hadamard gates placed at the start of each address qubit ensure all addresses are calculated simultaneously, but shot counts scaling with the number of addresses are required to ensure accurate reconstruction of the data.

NEQR

NEQR[22] builds on FRQI by adjusting the encoding of image values to be more accurate. Rather than using angle encoding to encode the value in one data qubit, it uses basis encoding by encoding an n -bit color value into n data qubits. This allows for a much more accurate recovery of color data, but sacrifices circuit compactness due to the added data qubits.

1.4 QSobel

Zhang et al. [21] proposed a quantum-augmented version of the Sobel operation named QSobel. Based on the angle encoding and addressing scheme used in FRQI, the study proposes the basis for a Sobel operator that leverages the advantages of quantum systems. It proposes to handle the calculation of the Sobel Filter values G using a quantum circuit by having each pixel simultaneously adding or subtracting its neighboring values. It does this through the use of shifted images. An example of this operation and its use case is seen in Fig. 1.3, which a region of pixels in a hypothetical image. By performing a shift operation on the original image, each address for the shifted image now has the information of its left neighbor.

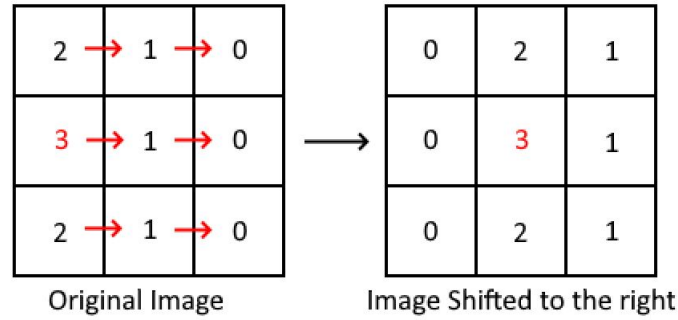


Figure 1.3: An example of how a right shift operation produces a new image where each address has the value of its left neighbor.

Thus, to calculate Gx or Gy , six shifted images that provide information on the neighboring pixels that have a non-zero kernel scalings are encoded into their own data qubits. By adding the values encoded in these qubits together for each address, Gx and Gy can be calculated simultaneously. Zhang et al. then proposes that these values can be squared and added together, with the square root of the final sum providing the Sobel values G for each pixel location.

QSobel has served as the foundation for newer methods that attempt to refine or expand its implementation[23], as the original method has some limitations. Its proposed method of shifting images within a quantum circuit makes use of a copy and store operation, but no such method exists in the quantum computing paradigm [19]. Additionally, when these papers were published, no methods for arithmetic on values encoded inside quantum circuits were developed, so those operations were treated as black-box sub-circuits. A viable means of shifting and storing images, quantum or otherwise, as well as a means for quantum

arithmetic, are needed to make the stenciling portion of the Sobel Filter implementable with quantum circuits. While no method for shifting and copying images in a quantum circuit exists and thus need to be handled classically, more recent work in the form of EHands [3], which will be discussed in the next subsection, has provided a method for quantum arithmetic.

1.5 LBNL Quantum Algorithms

To realize some of the missing pieces in Zhang’s original QSobel paper, this project makes use of several concepts and circuits developed by the Lawrence Berkeley National Labs.

QPixl

QPIXL [1] is framework of circuit creation that is compatible with FRQI for the purpose of encoding images. Similar to base FRQI, circuits using the QPIXL framework encode values through angles in Ry gates. While the range of values for the Ry gates is limited to the range 0 to $\frac{\pi}{2}$, this can be circumvented by scaling the original values down to the correct range using the formula

$$\theta_k = \frac{\pi/2}{K} g_k, \quad (1.6)$$

where K is the maximum value of the inputs and g_k is the value of the pixels. For

decoding the g_k values, the formula

$$g_k = \frac{255}{\pi/2} \arctan\left(\frac{\beta_k}{\alpha_k}\right) \quad (1.7)$$

is used, where $\alpha_k^2 = \cos^2(\theta_k)$ represents the probability of $|0\rangle$ and $\beta_k^2 = \sin^2 \theta_k$ represents the probability of $|1\rangle$.

The QPIXL framework builds on previous improvements in FRQI circuit design to minimize the amount of 1– and 2–qubit gates needed to represent the uniformly controlled rotation circuits. These circuits only require n Ry rotation and n CNOT gates for an image with n pixels, providing a quadratic improvement in gate complexity compared to the original FRQI circuits. QPIXL’s efficient image encoding is done by encoding each input angle as an equivalent series of angles across several Ry gates arranged in patterns that emulate solvable linear systems of equations. By solving the linear systems for each input, the fixed number of Ry and CNOT gates can encode all inputs. These linear systems can be solved as matrices. Alternatively, a variant of the Walsh-Hadamard transform [11] algorithm, often used for signal transformations, can be used to convert the original angles into UCRy circuits. The circuits can be further simplified by removing rotation gates with negligible impact on the results, determined by some compression criterion. After the removal of unnecessary Ry gates, any CNOT gates with the same control qubit in the same group between the remaining Ry gates should be canceled.

QCrank

QCrank[4] builds on the advances of QPIXL by allowing the Uniformly Controlled Ry sub-circuits developed in QPIXL to encode multiple data qubits simultaneously in a matter similar to MCRQI[17], a multichannel color version of FRQI, but much more efficiently. Simultaneous encoding is done through parallel uniformly controlled rotation gate (pUCRy) circuits. In this implementation, one cycle of the pUCRy sub-circuit with n parallel data qubits is logically equivalent to a normal serial UCRy circuit that is n times longer. Although the immediate obvious use case for this improvement is efficient encoding of multicolor channel images, this new capability can also be utilized to encode the multiple shifted images required for QSobel's simultaneous access of neighboring values during the stenciling calculations. A requirement for the version of QCrank used in this project is that the number of pixels in the input images must be a power of 2. As a result, the sizes of the images in the experiments in Sec. 3.2 are constrained to be powers of two in size.

EHands

EHands[3] is a novel encoding, decoding, and quantum arithmetic operation scheme. It includes the Expectation Values ENcoding (EVEN) scheme for encoding and decoding data and four sub-circuits for arithmetic compatible with EVEN encoded values: addition (more specifically weighted sum), multiplication, negation, and parity flips. The input values x_i from a source image, scaled down to the range $[-1, 1]$ if necessary, are encoded in Ry rotations

where $\theta_i = \arccos(x_i)$ applied to qubits initialized to the $|0\rangle$ state and decoded by measuring the Expectation Value (EV) for the relevant data qubits.

For details of EVEN, consider a 2-qubit quantum state initialized in $|00\rangle$. When Ry gates $Ry(\theta_0)$ and $Ry(\theta_1)$ are applied to their respective qubits, they create the following state:

$$|\phi\rangle = [Ry(\theta_0) \otimes Ry(\theta_1)] |00\rangle = \begin{bmatrix} \cos \frac{\theta_0}{2} \cos \frac{\theta_1}{2} \\ \cos \frac{\theta_0}{2} \sin \frac{\theta_1}{2} \\ \sin \frac{\theta_0}{2} \cos \frac{\theta_1}{2} \\ \sin \frac{\theta_0}{2} \sin \frac{\theta_1}{2} \end{bmatrix} \quad (1.8)$$

This formula could be further simplified by replacing θ_i with $\arccos x_i$, and using the half angle formulas for cosine and sine, leading to the equation

$$\begin{bmatrix} \sqrt{\frac{1+\cos(\arccos x_0)}{2}} \sqrt{\frac{1+\cos(\arccos x_1)}{2}} \\ \sqrt{\frac{1+\cos(\arccos x_0)}{2}} \sqrt{\frac{1-\cos(\arccos x_1)}{2}} \\ \sqrt{\frac{1-\cos(\arccos x_0)}{2}} \sqrt{\frac{1+\cos(\arccos x_1)}{2}} \\ \sqrt{\frac{1-\cos(\arccos x_0)}{2}} \sqrt{\frac{1-\cos(\arccos x_1)}{2}} \end{bmatrix} = \frac{1}{2} \begin{bmatrix} \sqrt{1+x_0} \sqrt{1+x_1} \\ \sqrt{1+x_0} \sqrt{1-x_1} \\ \sqrt{1-x_0} \sqrt{1+x_1} \\ \sqrt{1-x_0} \sqrt{1-x_1} \end{bmatrix} \quad (1.9)$$

The values can be decoded by measuring qubits 0 and 1 in the Z-basis, represented by

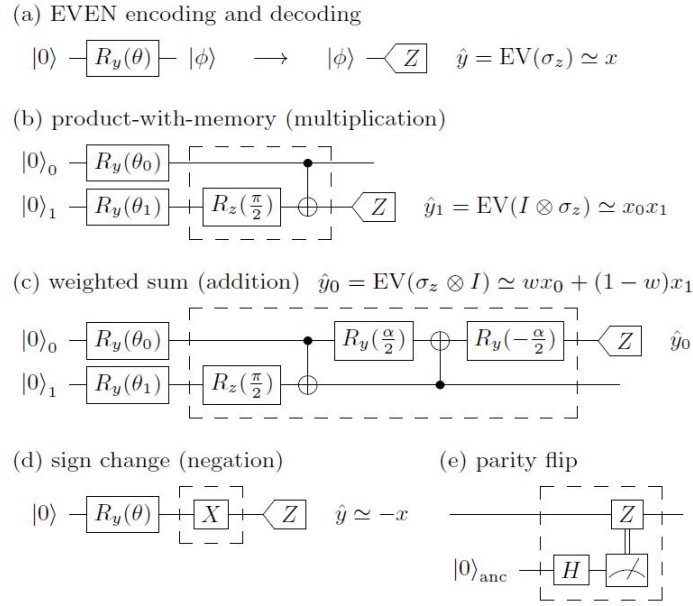


Figure 1.4: The initial versions of the EHands (a) encoding and decoding (b) multiplication (c) weighted sum (d) negation, and (e) parity flip sub-circuits [3]

their respective complementary operators

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix} \text{ and } \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix} \quad (1.10)$$

By applying these operators to the quantum state found in Eq. 1.8, the Expectation Value of the qubits will be x_0 and x_1 , respectively.

The first versions of the EHands sub-circuits are shown in Fig. 1.4. Circuit implementations will be discussed in detail in Sec. 2. The multiplication sub-circuit takes two encoding

qubits and stores their product in one of the qubits, with the other qubit retaining its origin value. The weighted summation sub-circuit computes the weighted sum $S_w(x_0, x_1) = wx_0 + (1 - w)x_1$. The weight w takes values from the range $[0,1]$ and controls the Ry gates used in the circuit using formula $\alpha = \arccos(1 - 2w)$. The negation sub-circuit changes the sign of the real input. The parity flip sub-circuit is utilized to remove an unwanted term created when weighted summation sub-circuits are performed sequentially. For example, when using two sequential weighted summations to add together 3 input values x_0 , x_1 , and x_2 , with weight w_1 , w_2 produces the EV $w_1[w_0x_0 + (1 - w_0)x_1] + (1 - w_1)x_2 + f(x_0, x_1, x_2; w_0, w_1)$, where the final term in the summation is unwanted. By applying the parity flip sub-circuit between the two weight sum sub-circuits, the EV $w_1[w_0x_0 + (1 - w_0)x_1] + (1 - w_1)x_2 - f(x_0, x_1, x_2; w_0, w_1)$ has a statistically equal chance of occurring, canceling out the unwanted term.

The original implementation of EHands circuits only had the first three sub-circuits reversible. However, this thesis developed an alternative reversible parity flip sub-circuit without mid-circuit measurements, making EHands fully reversible. This is not only relevant to the implementations used in this paper as it affects the type of simulator used in simulations, but it allows for a wider range of applications for EHands, such as using it in the oracle sub-circuits in Grover's Algorithm to tag elements [2].

Details regarding the actual implementation of the various methods, including the various iterations of the parity flip sub-circuit, will be discussed in detail in Sec.2.

1.6 Qiskit

Simulation of the circuits discussed in this paper were done using IBM’s qiskit software development for Python². In addition to functionality to design quantum circuits and simulate their functions, it provides interfacing with real quantum machines from IBM Quantum. This is done by transpiling, or converting, generalized circuit developed with qiskit into an equivalent architecture that is compatible with real IBM QPUs.

Simulators and Simulation options

Real QPUs were not available during the development of this thesis, so classical hardware needed to be utilized. In order to imitate quantum operations on classical hardware, simulators were utilized. IBM quantum provides the Qiskit Aer module to simulate how quantum circuits would behave on IBMQ QPUs [9]. To speed up operation, Aer allows for MPI and GPU assisted parallel processing.

Qiskit’s AerSimulator offers several different methods of simulation. The two simulator methods relevant to the circuits used in this paper are the ”statevector” and the ”density_matrix” simulator modes. Both are compatible with GPU acceleration, which helps minimize runtime. The statevector method for the AerSimulator works by calculating the state vector for the quantum circuit, then using the probability amplitudes to produce output counts that reflect them. In this implementation, the calculation of the a circuit’s state vec-

²Qiskit documentation online at <https://docs.quantum.ibm.com/>, May 2025

tor takes a significant portion of the simulation time, with the shots designated by the `run()` function distributed based on probability. The density matrix simulator behaves differently, and is used when gates such as reset or measurement gates are used mid-circuit [8]. These gates do not have unitary operations and thus not reversible, so a single state vector cannot be calculated at the end of the circuit due to the circuit being in a mixed state. Because the state vector is no longer guaranteed, the simulation has to run the circuit for each shot to get each outcome. Simulating each shot makes the total job runtime for density matrix simulation much more affected by shot count compared to the state vector simulator. There are other useful options in the form of the `seed_simulator` option, which allows for reproducibility of results, and `device`, which sets the simulator to either strictly CPU simulation or GPU accelerated simulation.

Parallelization options

Qiskit allows for some fine tuning of parallelization settings in the form of the `enable_blocking` and `blocking_qubits` parameters in the `run()` function, however these features for jobs utilizing either multiple GPUs or multiple CPU communicating via MPI protocol. Qiskit also allows for simultaneous execution of jobs in a batch. For the experiments in this project, neither feature was utilized, but analysis of their effectiveness can be topics of future work.

Chapter 2

Implementation

This chapter focuses on the quantum implementation of the quantum-augmented Sobel computation, through its use of QCrank and EHands operations. Because EHands does not have a circuit block that can conduct square root operations on data encoded within a quantum circuit, at least a portion of the complete Sobel computation must be performed classically. Details on the effect of moving from quantum to classical computations at several points in the implementation are discussed in the following section. The experiments in this thesis also include a verification step that compares the results of quantum and classical Sobel computations.

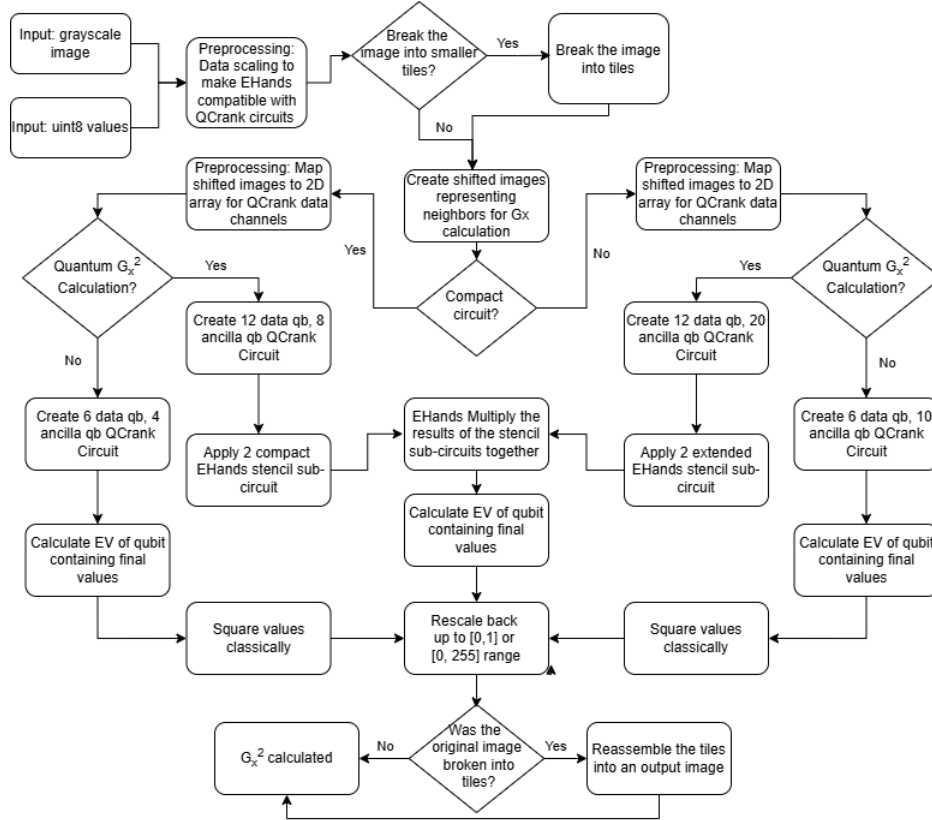


Figure 2.1: A flowchart of the overall implementation of just the G_x and G_x^2 calculations. A G_y and G_y^2 calculation flowchart would be similar, and the full $\sqrt{G_x^2 + G_y^2}$ was too large of a circuit to simulate on classical hardware. Additionally real QPUs were unavailable during testing, so the full Sobel gradient calculation needed to be done piecemeal using smaller quantum circuits

2.1 Overall Sobel Circuit Architecture

There are several variations of the quantum-augmented Sobel operation that were implemented and tested in this paper. Although the designs may differ, the end goal of calculating the Sobel gradient magnitude G , where $G = \sqrt{G_x^2 + G_y^2}$, is the same for all implementations.

Fig. 2.1 shows a flow chart that highlights the divergence points in the Sobel process that

lead to each of the circuit variants for the calculation G_x^2 . A flow chart for the G_y^2 calculation would be similar, the primary difference being the shifted images that are being loaded into the QCrank data channels. The G_x circuits load in shifted images that allow access to the values in the top right, right, bottom right, top left, bottom left, and bottom left neighbors of each address, while the G_y circuits load in the shifted images needed to access the values of the neighbors of the top left, top, top right, bottom left, bottom and bottom right of each address. Specifics regarding the mapping will be discussed in the subsections for the extended and compact stencil computation circuits.

In addition to conciseness, the scope of the flow chart is just the calculation of G_x^2 because it only shows what has been implemented and tested so far. This thesis also developed an implementation for a fully quantum $G_x^2 + G_y^2$ circuit, but could not simulated it due to the circuit's minimum width exceeding the maximum size for simulation of the machines used. The circuits could still fit within some QPUs provided by IBM Quantum for smaller image, but access to quantum hardware was not available during the writing of this paper. Only individual G_x , G_y , G_x^2 , G_y^2 circuits could be run on simulators, with the final summation of squared gradients G_x^2 and G_y^2 and the square root operation conducted classically. Further analysis of memory requirements is found in Sec. 3.2.

Table 2.1 shows the trade offs between quantum and classical computations. For the purposes of maximizing the amount of Sobel calculations converted into quantum circuits, the final point to transition back to classical is ideal. However, the final calculation cannot be simulated due to the large number of data qubits required to represent it. Two G_x and

Point of Transition back to Classical	Tradeoffs
After G_x or G_y	Circuit's data qubit count is kept at base size, but squaring, addition, and square root operation for each pixel must be handled classically for both calculations.
After G_x^2 or G_y^2	Circuit's data qubit count is doubled, but only the gradient addition and square root operation for each pixel must be done classically for both calculations.
After $G_x^2 + G_y^2$	Circuit's data qubit count is quadrupled, but only the square root operation for each pixel must be done classically. One circuit handles both gradients.

Table 2.1: The tradeoffs of moving the Sobel calculation back to classical computing at various points in the process.

two G_y circuits must provide their own sets of data qubits in a single circuit, quadrupling the amount of data qubits compared to the base stencil operation circuits. This circuit could potentially fit onto actual quantum hardware, subject to how the circuit is transpiled and the available QPU's layout and size. This is an important topic for future work that will be conducted when QPUs are available for testing.

Data Encoding: Interfacing EHands with QCrank

Although EHands can work directly with the data qubits of QCrank circuits, there are some steps needed to ensure that the circuit behaves correctly. The image pixel values first need to be converted to values that would normally be directly encoded as angles in Ry

gates using the normal EHands protocol. Classical operations create the six shifted images for the Sobel stencil operation, with a driver function inserting them into a numpy array of shape (image size, 6, 1) to be compatible with QCrank's *bind_data()* function. The third axis of this array is used to create multiple circuits, but this feature was not utilized for the implementations in this paper. QCrank then takes the modified input values and encodes them into a pUCRy circuit. A quantum circuit creation function then applies the Sobel stencil operation via the EHands operations. Details regarding the implementation of each EHands circuit block and EHand's EVEN encoding and decoding are found in Sec. 2.2.

The version of QCrank used in this project used an older encoding scheme in which the values were scaled down to the range $[0,1]$. QCrank provides a *max_val* variable as a parameter for circuit creation, which allows QCrank to calculate the correct scaling values to bring the values to the correct range. EHands EVEN encoding also takes inputs with a maximum value of 1. However, because the arccosine of these values are what is ultimately encoded into the Ry angles of the circuit, the range of values encoded by QCrank is instead $[0, \pi]$. The *max_val* variable is set to π as a result.

To summarize, this implementation of the Sobel Filter classically converts input values into EHands compatible values, classically creates 6 shifted copies of the image, encodes the values into a multi-channel circuit efficient in gate count and depth with QCrank, and applies the stencil operation on the data qubits with EHands. With this process, this implementation calculates all pixel values of an image simultaneously.

Extended Circuit Variant

The extended circuit implementation referenced in Fig. 2.1 refers to a more straightforward, yet less space-efficient version of the individual gradient calculation. This version more closely follows the circuit layout proposed in the QSobel paper[21]. The 6 data qubits are assigned to neighboring pixels that have non-zero kernel values applied to them. The mapping of pixel neighbors to data qubits is found in Fig. 2.2.

-1 qb 5	0	1 qb 2	-1 qb 3	-2 qb 4	-1 qb 5
-2 qb 4	0	2 qb 1	0	0	0
-1 qb 3	0	1 qb 0	1 qb 0	2 qb 1	1 qb 2

a) G_x
b) G_y

Figure 2.2: How each neighbor of a pixel in the G_x and G_y kernel is mapped to one of the 6 data qubits in each of the extended Sobel's stencil circuits.

Fig. 2.3 shows the sub-circuit needed to calculate the Sobel convolution with this layout. This circuit structure works with both G_x or G_y , depending on the mapping used in Fig. 2.2. Details regarding how the shifted images are created and mapped to QCrank data qubits are discussed in the sub-section Data Preprocessing: Shifted Images, found later in this chapter. 10 total ancilla qubits, 6 for scaling and 4 for parity flip gates, are utilized by each G_x and

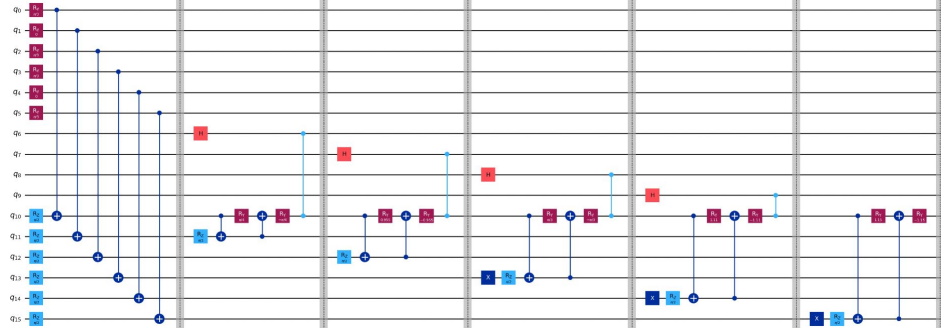


Figure 2.3: The circuit diagram of the extended Sobel convolution portion of the quantum circuit. The QCrank data encoding sub-circuit located to the left of the circuit and address qubits located below the circuit were left out for space. Note the negations placed on last 3 blocks, which represent subtraction between the positive and negative portions of the filter kernel. Note that the barriers included here are for readability and removed during the execution of the circuit

G_y calculation circuit. The 6 least significant bits are the ancilla qubits that store scaling values applied to the data qubits via separate EHands multiplication blocks. EHands accepts an input range of -1 to 1, so this implementation scaled the weights so that the maximum value of the strongest edges does not exceed EHand's numerical limit. The resulting stencil matrices are produced for G_x and G_y , respectively, to work within that constraint.

$$\begin{bmatrix} -0.5 & 0 & 0.5 \\ -1 & 0 & 1 \\ -0.5 & 0 & -0.5 \end{bmatrix} \text{ and } \begin{bmatrix} -0.5 & 1 & 0.5 \\ 0 & 0 & 0 \\ -0.5 & 1 & 0.5 \end{bmatrix} \quad (2.1)$$

The last three weighted summations in the circuit are negated, effectively subtracting the negative portion of the Sobel filter window.

Although intuitively it might seem that the ancilla qubits providing 1x scaling do not

have an impact on the circuit, they are actually needed for the correct computation of the final value. A plain average over K variables was used[3], defined by the formula:

$$\frac{1}{K} \sum_{i=0}^{K-1} x_i \quad (2.2)$$

with weight $w_i = \frac{i+1}{i+2}$ for $i \in [0, \dots, K-2]$. In the case of this implementation, the weight values produced are $1/2$, $2/3$, $3/4$, $4/5$, and $5/6$. While intuitively the 1x scaling on qubits qb 1 and qb 4 don't seem like they would have an impacts on calculation, for this formula to work with scaled values, all inputs must be scaled to produce a new set of variables $x'_0, x'_1, \dots, x'_5$, where $x'_0 = 0.5x_0$ etc. If some of the original values are applied directly to the plain average formula, the correct results will not be achieved.

To make use of the updated parity flip gate, four ancilla qubits are needed for the first four sequential weighted summation blocks needed to simulate the summation for G_x and G_y . The final summation does not need a parity flip to remove the unwanted quantum information, as it is not expressed in measurement. Because the signal values produced by this weighted sum are scaled down by $K = 6$ and the kernel window values are scaled down by 2, the final outputs must be classically scaled back by 12 to produce the same value as classic Sobel. However, due to the 2x scaling of the input values during data encoding, which will be discussed in the EVEN implementation in Sec. 2.2, the values produced only need to be scaled up by an additional factor of 6.

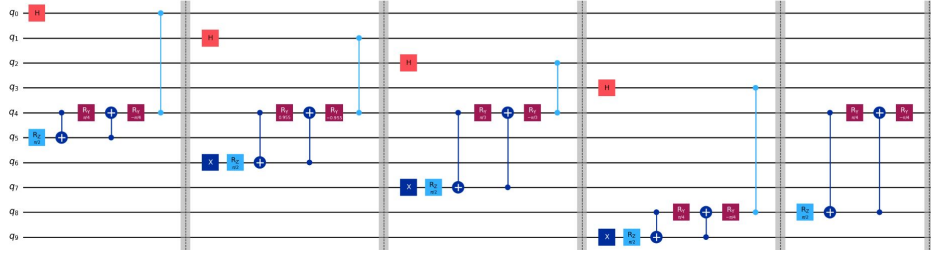


Figure 2.4: The circuit diagram of the compact Sobel convolution portion of the quantum circuit. As with the extended circuit diagram, the QCrank data encoding located to the left of the circuit and address qubits located below the circuit were omitted for brevity. Note the negations placed on middle 3 blocks, which represent subtraction between the positive and negative portions of the filter kernel. Note that the barriers included here are for readability and removed during the execution of the circuit

Compact Circuit Variant

This version of the quantum convolution operation, shown in Fig. 2.4 reduces the amount of required ancilla qubits by adjusting the weights utilized by the weighted summations to double the coefficients attached to the qubits associated with the higher-value portions of the kernel[2]. This removes the need for ancilla qubits that assist in stencil kernel scaling. The adjusted mapping is found in Fig. 2.5.

Rather than utilizing ancilla qubits to adjust data qubit weight values like in the extended variant, this circuit used more deliberately chosen weights to adjust the relative scaling of each qubit. Assigning weights $1/2$, $2/3$, $3/4$, $1/2$, and $1/2$ to the sequence of weighted sums

-1 qb 2	0	1 qb 0	-1 qb 3	-2 qb 5	-1 qb 2
-2 qb 5	0	2 qb 4	0	0	0
-1 qb 3	0	1 qb 1	1 qb 1	2 qb 4	1 qb 0

a) G_x b) G_y

Figure 2.5: How each neighbor of a pixel in the G_x and G_y kernel is mapped to one of the 6 data qubits in each of the compact Sobel's convolution circuits.

S1 to S5 results in the following formulas:

$$\begin{aligned}
 S1 &= \frac{1}{2}q0 + \frac{1}{2}q1 = \frac{q0 + q1}{2} \\
 S2 &= \frac{2}{3}S1 - \frac{1}{3}q2 = \frac{q0 + q1 - q2}{3} \\
 S3 &= \frac{3}{4}S2 - \frac{1}{4}q3 = \frac{q0 + q1 - q2 - q3}{4} \\
 S4 &= \frac{1}{2}q4 - \frac{1}{2}q5 = \frac{q0 + q1}{2} \\
 S5 &= \frac{1}{2}S3 + \frac{1}{2}S4 = \frac{q0 + q1 - q2 - q3 + 2q4 - 2q5}{8}
 \end{aligned} \tag{2.3}$$

This circuit variant also adjusted the QCrank data qubit assignments so that the values associated with the lower-magnitude kernel are next to each other on the least significant data qubits. This rearrangement does not have an impact on circuit function, but does create a cleaner circuit that stores EV values in the least significant data qubit when possible. The weighted summation S5 would contain the final value of the Sobel convolution calculation,

and, like the extended circuit, it has its value scaled down. The weighted summations in isolation bring probability amplitude to $1/8$ of the intended value, but like with the extended circuit, the implicit $2x$ scaling of during the data encoding step makes the final values $1/4$ of the intended values.

Parity Flips with Reset

For both compact and extended circuit variants, the number of ancilla qubits for the weighted summation sequence can be reduced from 4 to 1 when resets are implemented. However, reusing ancilla qubits sacrifices circuit reversibility, making it equivalent to the original version of the parity flip block that utilized mid-circuit measurements. Additionally, utilizing reset gates force the simulator to utilize the slower Density Matrix simulator which simulates each shot. As a result, this variation was not simulated.

Data Preprocessing: Shifted Images

This section covers how this implementation creates shifted images with classical computing methods, as well as the mapping of the images to QCrank. The method of shifted images follows the intended behavior outlined in the QSobel subsection of Sec. 1.3. However the shifting is implemented classically rather than within the quantum circuits because the original method proposed by Zhang et al. in their QSobel[21] paper used an unproven quantum copy operation. Because the process is similar between the extended and compact variants

of the Sobel stencil circuits and both stencil operations, this section will only explain the G_x circuit. In order for all pixels in an image to gain information from a specific neighbor pixel, a shift operation must be conducted in the opposite direction, so that neighbor's value is sitting at the coordinates of the original pixel. This is accomplished trivially with the numpy `roll()` operation. This shifted image representing one of the 6 data qubits is then flattened and inserted into a numpy array. QCrank reads these channels as with the assumption that the most significant qubit is first, so channels are added to the numpy array in descending order. The implementation for creating shifted images that will map to the extended G_x calculation circuit is found in Listing 2.1.

```

1 def build_gx_array(image):
2     # qb5 - top left: (Y+1, X+1)
3     temp = np.roll(image, (1, 1), axis=(0, 1))
4     gx = temp.reshape(-1,1)
5     # qb4 - left neighbor: (Y, X+1)
6     temp = np.roll(image, 1, axis= 1)
7     gx = np.append(gx, temp.reshape(-1, 1), axis=1)
8     # qb3 - bottom left neighbor: (Y-1, X+1)
9     temp = np.roll(image, (-1, +1), axis=(0, 1))
10    gx = np.append(gx, temp.reshape(-1, 1), axis=1)
11    # qb2 - top right neighbor: (Y+1, X-1)
12    temp = np.roll(image, (1, -1), axis=(0, 1))
13    gx = np.append(gx, temp.reshape(-1, 1), axis=1)
14    # qb1 - right neighbor: (Y, X-1)
15    temp = np.roll(image, -1, axis=1)
16    gx = np.append(gx, temp.reshape(-1, 1), axis=1)
17    # qb0 - bottom left neighbor: (Y-1, X-1)
18    temp = np.roll(image, (-1, -1), axis=(0, 1))
19    gx = np.append(gx, temp.reshape(-1, 1), axis=1)
20    return gx

```

Listing 2.1: The implementation for generating the shifted images required for the G_x calculation. The original image is shifted in the opposite direction of the neighbor it needs information from.

An issue with roll operations is that values that get shifted out of the image range

wrap back to the other side of the image. This results in issues with Sobel calculations at the borders of an image, as some of the neighboring values are not valid. This issue is not unique to this implementation, many stencil operations using kernel windows must make special cases of border regions. Although there are other ways to handle these border regions, such as replacing the wrapped values with one placeholder value to reduce the impact of missing valid data, border regions cannot have fully valid kernel windows. For classical implementations, these regions can be avoided by simply adjusting the range of coordinates to apply the Sobel stencil calculations, but quantum stencil operations cannot define specific ranges. As a work around, for the final Sobel image output, the border coordinates have their values set to 0.

Gradient Squaring Calculation

There is no direct EHands circuits for squaring values. However, this can be circumvented by implementing two identical Sobel stencil operations in one circuit and taking the product of their outputs. A squaring operation doubles the total non-address qubits for non-reset gate implementations. Because this requires a large minimum memory requirement which will be discussed in further detail in Sec. 3.2, only the compact stencil variant was used for G_x^2 and G_y^2 calculations. The implementation of the compact squaring circuit can be found in Fig. 2.6. As with the previous extended stencil operation sub-circuit figure, this one does not feature the large QCrank sub-circuit before it nor the address qubits below it.

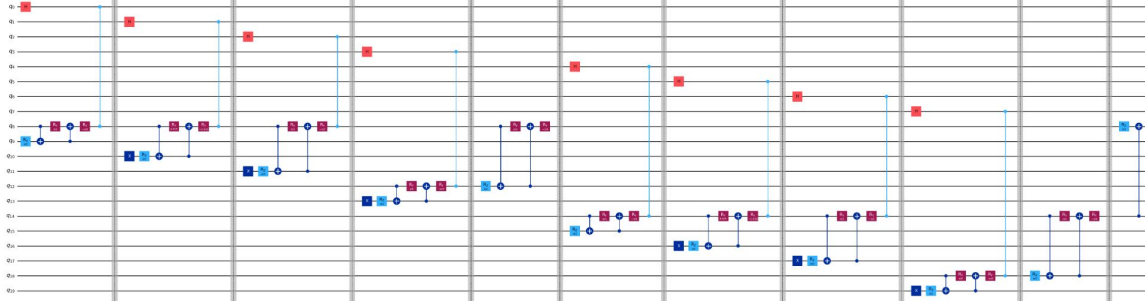


Figure 2.6: The implementation of a G_x^2 or G_y^2 calculation circuit, which uses 2 G_x or G_y circuits stacked on top of each other, with their results multiplied together. Note that the barriers included here are for readability and removed during the execution of the circuit

Data Preprocessing: Image Tiling

Image tiling is a primarily classical optimization for this project and currently mainly exhibits benefits with simulation execution, but it also has potential benefits with executions on real QPUs, which will be discussed in Sec. 3.8. The implementation itself involves breaking an image into smaller tiles that are still valid inputs for QCrack. For the square images used in the experiments, the simplest way to do this is to divide the image into quadrants, producing four square tiles. These tiles can then be broken into quadrants, generalizing the valid tile counts to be powers of 4, so long as the tiles are of length 8 or larger. This is because a tile length of cannot produce any values where all of its neighbors are valid non-border addresses.

While this tiling method allows for multiple smaller circuits to handle the Sobel calculations, potentially simultaneously if there are enough real QPUs available in a non-simulation environment to run parallel jobs, more must be done to achieve an accurate result. Fig. 2.7

shows the results if only 4 tiles are used to handle the Sobel calculation. The blue shaded regions in the fourth image show that the issues that occur when handling the full image's border values is also present at the borders of each tile.

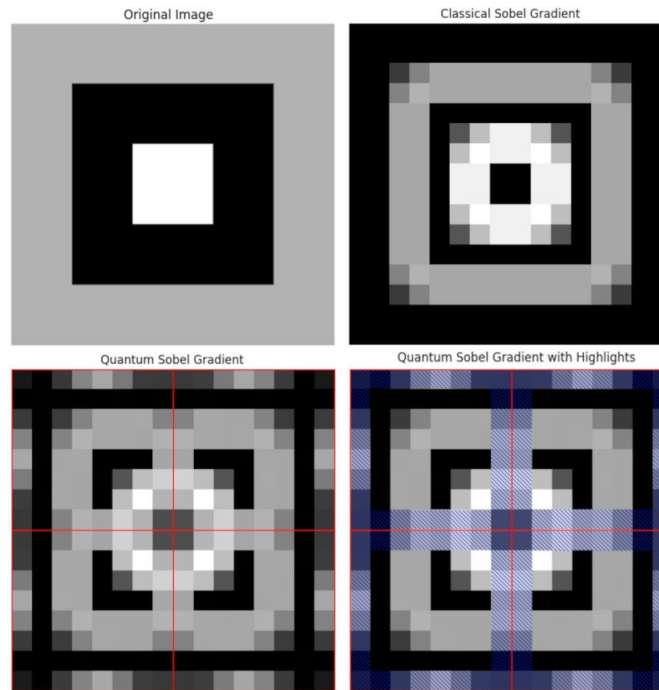


Figure 2.7: Comparison of the expected Sobel result, which was computed classically, with the combined results of 4 tiles of quantum-augmented Sobel made up of the quadrants of the original image. The images themselves are grayscale, with the red lines added to better show the area of each tile. The fourth image highlights the tile borders, which are not calculated correctly because not all of their neighbors used in their Sobel stencil operations are valid.

At minimum, any increases in tile length will require doubling a dimension because QCrank only takes pixel values of powers of two as noted in Sec 1.5. Because small adjustment to tile sizes are incompatible with QCrank, more tiles were added to allow for overlap instead. With an implementation with no overlap, a tile's shape can be defined as

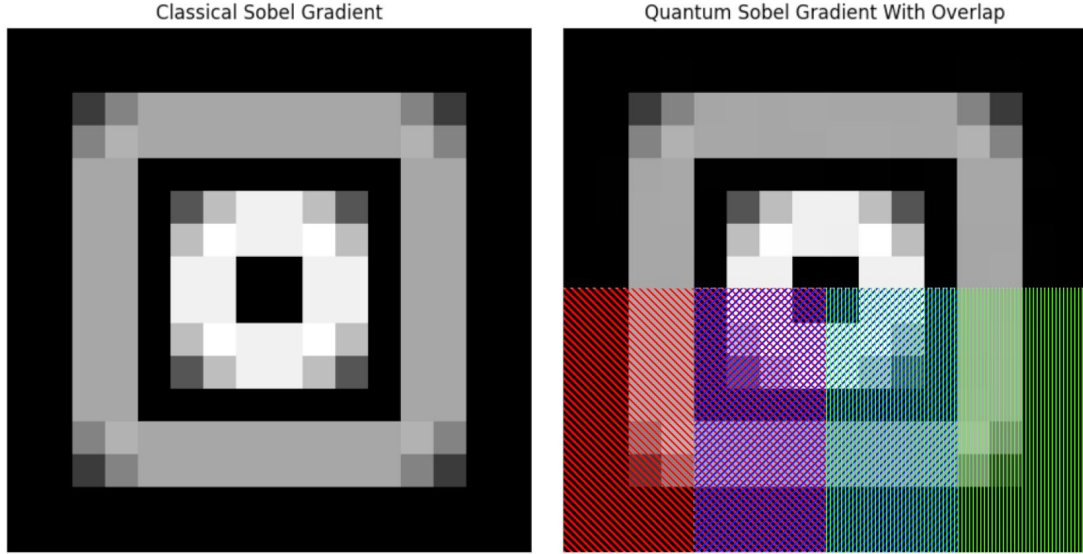


Figure 2.8: Comparison of the expected Sobel result, which was computed classically, with the combined results of 9 tiles of quantum-augmented Sobel made up of the quadrants sized tiles of the original image. The images themselves are grayscale, the red, blue, and green shaded boxes representing the tiles used for the bottom row of the image

$n \times n = N/m \times N/m$, where N is the length of the overall image and m is the number of tiles needed to fill one dimension of the image without overlap. From this, m^2 tiles are needed in total to full reconstruct an image. To incorporate overlap, $(m + 1)^2$ tiles are needed instead, with an additional tile used in each dimension. The additional tile length in each dimension is evenly distributed among each tile, so $n * \frac{1}{m}$ pixels of each tile can be used as overlap. The overlapping technique is applied for the input image used in Fig. 2.7, providing the results shown in Fig. 2.8. The image is broken up into 9 tiles. The colored boxes represent the bottom row of 3 tiles, with regions with two colors showing which portions of the bottom half of the image are experiencing overlap.

With this implementation providing at most 1 extra tile for overlap, there is an issue when the number of the tiles is equal or greater than the length of each tile. With only one tile's length worth of overlap and $n = m$, only $m * \frac{1}{m} = 1$ pixel of overlap is allotted for each tile, which would require the use of tile border values. However, as discussed in the following tile stitching subsection, this implementation only inserts non-border addresses to the final image, so rather than utilize the invalid pixel values, these regions would instead be skipped when values are applied to the final image. This keeps them at their default initialized value of 0. To fix this, more extra tiles would be required. This issue has not yet been addressed in a reliable manner and will be the subject of future work.

The code for achieving the overlap effect without addressing the issue discussed above is found in the Listing. 2.2. The *overlap* variable is first calculated by dividing the tile length *tile_len* by the number of tiles that can fit in one dimension of the image without overlap *n_tiles_1D*. This produces the number of addresses in one dimension that a tile would share with another tile. The *stride* variable then calculates when to start a new tile within an existing tile by subtracting the overlap value from *tile_len*. The *starts* list translates the stride into coordinates in the overall image.

Data Postprocessing: Tile Stitching

When Sobel calculations utilize overlapped tiling, the final image reconstruction is essentially the reverse of the tiling process. The details of the code execution are found in

```

1 def tile_image_overlap(img, n_tiles):
2     n_tiles_1D = int(math.sqrt(n_tiles))
3     overlap = tile_len // n_tiles_1D
4     stride = tile_len - overlap
5     tiles = []
6     starts = [int(i*stride) for i in range(n_tiles_1D+1)]
7     for i in range(len(starts)):
8         for j in range(len(starts)):
9             y = starts[i]
10            x = starts[j]
11            tiles.append(img[y : y + tile_len, x : x + tile_len].copy())
12 return tiles

```

Listing 2.2: Code for generating overlapping tiles. It does not contain code to address the cases where 1 or less addresses can be distributed given only one extra tile of overlap.

Listing 2.3. A tile list is received as an input and the regions of the final image are built in the same order as the tile creation conducted on the original image. The relevant variables needed to calculate *overlap* and *stride* are recreated here. For each tile the region of the final image that it belongs to is calculated and the non-border pixel values are used to fill out the region. When filling out a pixel that was populated by a previous tile, the old value is replaced. An optimization on the classical side that could potentially reduce error is averaging the values assigned to each addresses in overlap regions rather than replacing old values when a new tile assigns its values. The effectiveness of this optimization the potential subject of future work.

Establishing Ground Truth Values: Classical Sobel

To generate a ground truth to compare the accuracy of the quantum-augmented results, a simple fully classic Sobel operation was created to implement the intended operations of

```

1 def reconstruct_from_tiles(tiles, n_tiles):
2     n_tiles = len(tiles)
3     n_tiles_1D = int(math.sqrt(n_tiles)-1)
4     tile_len = tiles[0].shape[0]
5     image_len = int(n_tiles_1D * tile_len)
6     overlap = tile_len // n_tiles_1D
7     stride = tile_len - overlap
8     reconstr_img_list = np.zeros((image_len, image_len))
9     for i in range(n_tiles_1D + 1): # Include the extra tile for overlap
10        for j in range(n_tiles_1D + 1):
11            y = i * stride
12            x = j * stride
13            tile = tiles[i * (n_tiles_1D + 1) + j]
14            reconstr_img_list[y+1 : y+tile_len-1, x+1 : x+tile_len-1] =
                tile[1 : tile_len-1, 1 : tile_len-1]
15     return reconstr_img_list

```

Listing 2.3: Code for populating the final image with the quantum-augmented Sobel calculation tiles. The output image is initialized to a black image, and only the values of non-border addresses of each tile applied to the output picture. This implementation replaces values in addresses already populated by previous tiles.

the quantum-augmented operation. This fully classical approach only conducted operations present in the quantum-augmented approach to ensure the results could be comparable. The implementation of the full Sobel gradient magnitude calculation can be found in Listing 2.4, however, smaller versions were also created to test the G_x , G_y , G_x^2 , and G_y^2 circuits.

2.2 EHands Arithmetic

EVEN: EHands Encoding and Decoding

Pseudocode of the EHands encoding sub-circuit implementation is found in Listing. 2.5. Assuming 8-bit grayscale values g_i of range $[0, 255]$, they are scaled down to range $[-1, 1]$ with the formula $((g_i/255) - 0.5) * 2$. While the 2x scaling will need to be accounted for

```

1 def sobel_classical(image):
2     image = image/255
3     sobel_image = numpy.zeros_like(image)
4     for i in range(1, image.shape[0] - 1):
5         for j in range(1, image.shape[1] - 1):
6             gx=(image[i-1, j+1] + 2 * image[i, j+1] + image[i+1, j+1]) - \
7                 (image[i-1, j-1] + 2 * image[i, j-1] + image[i+1, j-1])
8             gy=(image[i+1, j-1] + 2 * image[i+1, j] + image[i+1, j+1]) - \
9                 (image[i-1, j-1] + 2 * image[i-1, j] + image[i-1, j+1])
10            sobel_image[i, j] = numpy.sqrt(gx**2 + gy**2)
11    return sobel_image

```

Listing 2.4: Classical implementation of the full Sobel Operation. For variants that test intermediate values such as gx and gy, the sobel image coordinate assignment is replaced with the relevant variable or calculation.

when converting the final values back to the appropriate range, because the stencil calculation calculates relative differences value, the subtraction by 0.5 has no impact on the final gradient value. For simplicity, the final output values are left at a scale of $[0, 1]$, as they produce the same results visually as if they were scaled back to $[0, 255]$.

```

1 # Assume data is an unsigned 8-bit grayscale value
2 adjusted_val = ((data/255)-0.5)/0.5
3 circ = QuantumCircuit(1)
4 circ.ry(numpy.arccos(adjusted_val))
5 circ.measure_all()

```

Listing 2.5: Implementation of EHands Encoding

To accurately estimate the EV values, this implementation must run its circuits repeatedly until the deviation of the results is small relative to the range of values extracted[4]. The standard deviation for the EV follows the formula $\sigma = N_s^{-\frac{1}{2}}$, where N_s is the number of shots per address[4]. 5000 shots per address was chosen, as it provides an expected standard deviation of $1/\sqrt{5000} = 0.014$, which produces a range of errors almost unnoticeable for the

scaled down output image value range of $[-4/6, 4/6]$ for the extended circuit and $[-4/4, 4/4]$ for the compact circuit. For these ranges, the extended circuit has $8/6 * (1/256) = 0.005$ between each grayscale value, while the compact circuit has $2/1 * (1/255) = 0.007$ between each value. The EV would effectively be only a couple values off if converted back into uint8 grayscale values. Larger values would provide diminishing returns for the EV standard deviation while having a greater impact on runtime, so 5000 was considered a good balance between accuracy and speed.

```

1 # target_qubit: str of the non-addr qbs; qb w/ EV set to 1, rest 0
2 def get_EV(counts, nq_addr, target_qubit = '1'):
3     length = int(2**(nq_addr/2)) # assumes square image
4     output_image = np.zeros((length, length), dtype=float)
5     for i in range(length**2):
6         #converts i to binary
7         upper_bits_str = bin(i)[2:].zfill(nq_addr)
8         c_0=counts.get(upper_bits_str+'0'*len(target_qubit))
9         c_1=counts.get(upper_bits_str+target_qubit)
10        if c_0 == 0 and c_1 == 0:
11            expectation_value = 0
12        else:
13            expectation_value = 1 - (2*c_1/(c_0 + c_1))
14        x_coords = i % length
15        y_coords = i // length
16        output_image[y_coords][x_coords] = expectation_value
17    return output_image

```

Listing 2.6: Assigning EHands EVs to addresses in the output image

The decoding function calculates the EV of each address. This function is shown in the Listing. 2.6. For each address, its equivalent binary value is stored in *upper_bit_str*, which represents the portion of each basis state that represents coordinates. The values c_0 and c_1 are then found. c_0 represents the sum of counts for basis states where the value of its EV-storing qubit is 0, and its address qubits match *upper_bit_str*. Similarly, c_1 represents

the sum of counts for basis states where the value of its EV-storing qubit is 1, and its address qubits match *upper_bit_str*. EV is finally calculated using the formula $EV = 1 - 2c_1/(c_0 + c_1)$, a modified version of the expectation value formula that uses counts instead of probabilities. By only measuring the data qubit containing the EV along with the address qubits, the measurement gates implicitly handle the summation of basis state counts needed to produce c_0 and c_1 . The counts are grouped by what basis states in the circuit's state vector are observable. If a qubit is not measured, the basis states representing when it is in $|1\rangle$ will not be displayed.

EHands Multiplication with Memory

Pseudocode of the EHands Multiplication with Memory circuit is found in Listing. 2.7. This implementation acts upon an already initialized circuit, and applies the multiplication block on qubits designated by the arguments multiplicand and multiplier. The multiplicand qubit will retain its original value, while the multiplier qubit will store the product of the values originally encoded in the qubits.

EHands Parity Flip

This thesis implemented two variants of the EHands Parity flip block, the original developed by Balewski et al. [3], and a novel reversible implementation developed by me for this project. Both variants are consolidated into one function shown in Listing 2.8, with

```

1 def ehands_product_with_memory(ckt, multiplicand, multiplier):
2     # Assumes phase rotations are already applied to the inputs outside of
3     # this function.
4     # ckt: quantum circuit to be modified
5     # multiplicand: addr of multiplicand qubit (value will be retained)
6     # multiplier: addr of multiplier qubit (product will be stored and
7     # measured from here)
8     ckt.rz(np.pi/2, multiplier)
9     ckt.cx(multiplicand, multiplier)
10    ckt.barrier()
11    return ckt

```

Listing 2.7: Implementation of EHands Multiplication with Memory. This implements the circuit between the multiplicand and multiplier qubits in the circuit ckt, and returns the updated version of it.

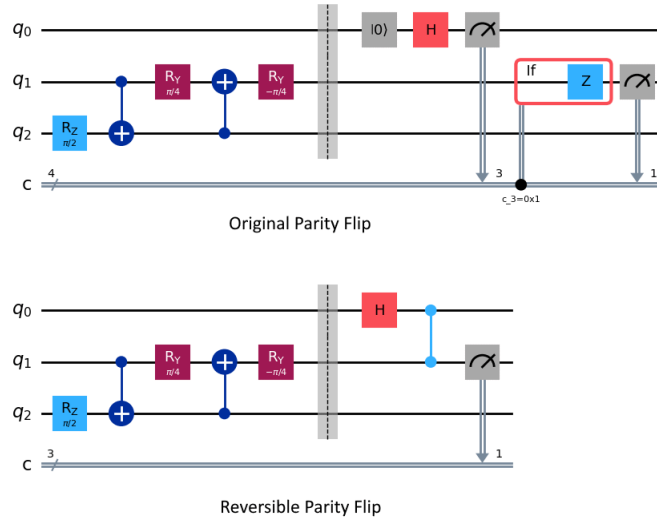


Figure 2.9: Circuit comparisons between the original EHands parity flip block and the reversible EHands parity flip block developed for this thesis. The mid-circuit measurement is replaced by a CZ gate.

comparisons of their circuit design seen in Fig. 2.9. The reset parameter controls which version is used. When reset is true, the original implementation developed by Balewski et

al. is inserted into the circuit.

The original implementation makes use of a reset gate and Hadamard gate to create equal possibility for outcomes of the ancilla qubit. Based on what is measured in the ancilla qubit, a conditional z rotation is applied to the target qubit that would house the unwanted quantum information discussed in Sec. 1.5. This conditional z rotation is represented by the red box in the first circuit of Fig. 2.9, which activates when the most significant classical bit, which stores the results of the mid-circuit measurement, is 1. By using reset gates, the original implementation has the benefit of only needing one ancilla qubit, as it is reset between use. However, by using mid-circuit measurements and reset gates, the original circuit is not reversible, thus preventing Qiskit from utilizing the more efficient state vector simulator. The density matrix simulator used instead creates much slower simulations, as the outcome of each shot must be simulated due the state of the circuit not being a fully known pure state. More importantly, it prevents EHands from being used in applications that require a fully reversible protocol.

The new, reversible variant of the parity flip, a contribution of this thesis work, intuitively accomplishes the same effect of applying a Z-flip operation to the target qubit, but does not require a measurement gate. All gate operations in this variant are reversible, which increases the range of potential applications for EHands. It also has the benefit of allowing simulations to run using the state vector simulator, which reduces the impact of shot counts during runs. A negative of this implementation is that in order to maintain its reversibility, it cannot use reset gates, thus requiring a unique ancilla qubit for each parity flip block.

```

1 def ehands_parity_flip(ckt, target, ancilla, cr, reset=False):
2     if reset:
3         ckt.reset(ancilla)
4         ckt.h(ancilla)
5         ckt.measure(ancilla, cr)
6         with ckt.if_test((cr, 1)):
7             ckt.z(target)
8     else:
9         ckt.h(ancilla)
10        ckt.cz(ancilla, target)
11    return ckt

```

Listing 2.8: Implementation of both the original and updated EHands Parity Flip operation. The former makes use of a mid-circuit measurement on the ancilla bit, and a conditional application of a Z flip based on the outcome of the measurement, while the latter is composed of just an h gate and a controlled Z gate

EHands Weighted Summation

```

1 def ehands_addition(ckt, addend1, addend2, weight, negation=False, ancilla
2     =-1, reset_parity=False):
3     alpha = numpy.arccos(1-2*weight)
4     if negation:
5         ckt.x(addend2)
6     ckt.rz(np.pi/2, addend2)
7     ckt.cx(addend1, addend2)
8     ckt.ry(alpha/2, addend1)
9     ckt.cx(addend2, addend1)
10    ckt.ry(-alpha/2, addend1)
11    if ancilla>=0:
12        ckt = ehands_parity_flip(ckt, addend1, ancilla, reset_parity)
13    ckt.barrier()
14    return ckt

```

Listing 2.9: Implementation of EHands Weighted Summation

Pseudocode of the EHands Weight Summation circuit implementation is found in Listing 2.9. Like the weighted summation circuit, it appends its respective EHands sub-circuit onto the end of an existing circuit's data and ancilla qubits. Weight values are passed in, and used to calculate the α value used in the Ry rotation gates. The option to add a EHands

negation to turn the weighted summation into a weight difference was added to make the circuit more versatile. The optional arguments *ancilla* and *reset_parity* are utilized when the EHands weighted sum block is part of a sequence of weighted sums and parity flip sub-circuits are needed to remove unwanted quantum information. If the *ancilla* argument contains a nonnegative value, the parity flip sub-circuit will be applied after the main summation sub-circuit, with the value provided denoting which qubit will serve as the ancilla qubit. The *reset_parity* argument controls whether the original parity flip version is used over the default reversible circuit.

Chapter 3

Evaluation and Results

This chapter covers the experiments, the software and hardware environment used to run them, and explanations of the performance metrics used to evaluate the results of each experiment. Experiment 1 verifies the effectiveness of each of the EHands circuits and the stencil circuits that fit within classical memory. Experiment 2 compares a scalability comparison between the compact and extended Sobel stencil circuit with increasing image sizes, analyzing runtime, accuracy, standard deviation, and circuit characteristics, such as shape and gate count. Based off the results of Experiment 2, the better performing implementation is used for later experiments. Experiment 3 compares the CPU node and GPU node runtime of the quantum portions of the full Sobel Filter gradient magnitude calculation and provided a sample result. Experiment 4 compares an implementation that used one circuit that calculates stencil operations on the whole image vs an implementation using series of smaller circuits to calculate quadrant-sized tiles of the original image. Experiment 5 analyses

the difference in runtimes of various tile sizes on a fixed-sized image.

3.1 Computational platform and Software

Environment

To test the effectiveness of the quantum-augmented Sobel implementations, this paper ran experiments using exclusive CPU and GPU nodes of the Perlmutter supercomputer at NERSC¹. The Perlmutter CPU nodes make use of two AMD EPYC 7763 CPUs², which has a 2.45GHz base clock rate, peak of 39.2 GFLOPS per core, and is made up of 8 Core Complex Dies (CCD), each consisting of 8 Zen3 cores. Each core has a 32KB L1 cache, 64B L1 cache line/block, 512KB L2 cache, and 256MB L3 cache that it shares with the rest of the chip. Perlmutter’s CPU nodes also have 512 GB of DDR4 memory total. Perlmutter GPU nodes each use a single AMD EPYC 7763 connected to four NVIDIA A100 GPUs³, all fully connected to each other. An A100 GPU is made up of 108 streaming multiprocessors, each with 64 warps per SM, and 32 threads per warp, totaling 22,184 threads. It has 192KB of L1 Cache per SM, 40MBs of L2 Cache, and 40GBs of second generation High Band Memory DRAM. Its maximum clock speed is 1410MHz. Both nodes use SUSE Linux Enterprise Server 15 SP4 for there OS’s.

¹Perlmutter architecture specifications online at <https://docs.nersc.gov/systems/perlmutter/architecture/>, Apr 2025

²EPYC documentation online at <https://www.amd.com/en/products/specifications/server-processor.html>, Apr 2025

³A100 documentation online at <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-nvidia-us-2188504-web.pdf>, Apr 2025

Regarding the quantum simulation software used, IBM’s qiskit software suite was used, specifically qiskit v1.2.2, qiskit-aer v0.15.1, qiskit-ibm-runtime v0.23.0. Qiskit’s Aer software was used for the ideal simulation of the quantum-augmented Sobel operator circuits using the state vector method, which has support for GPU based simulation. Regarding other major portions of the software stack, Python v3.12.3 was used, as well as numpy v1.26.4 to assist in the preprocessing and postprocessing of data in the classical computing portion of the implementation. Matplot v3.8.4 to generate most of the experiments.

3.2 Methodology

The focus of this paper is the development and analysis of the quantum portion of this Sobel implementation, so the benchmarking experiments will reflect this. Optimization of the classical portion of the implementation was not a priority, as it does not play a role in measurements. As a result, the image preprocessing steps, the quantum circuit building, and the latter portions of the gradient magnitude G calculations conducted using classical computing techniques are not monitored by benchmarking.

Performance metrics of focus for this paper are runtime, quantum circuit width, quantum circuit depth, gate count, and output accuracy. Because quantum runtime is the focus, the measured elapsed runtime will only cover the time it takes for the quantum simulation to run. Additionally, qiskit barrier()s, which aid in readability when displaying quantum circuits, have a negative impact on circuit depth and runtime by increasing both metrics,

and as a result are removed from the EHands stencil calculation portion of the circuit during testing. Circuit depth represents the number of layers of quantum gates executed in parallel needed to complete the computation of a quantum circuit. Circuit depth can be increased by barriers because they prevent optimizations in the form of simultaneous execution of gates that would otherwise be executed in parallel. This also causes a potential increase in runtime, as operations after a barrier must wait for all gates before a barrier to complete before running. Due to this, barriers are utilized only when displaying circuits for readability and are not used when measuring runtime and circuit depth.

Runtime

The pseudocode in Listing 3.1 shows the rough implementation, where the current time is recorded just before and just after the quantum circuit simulation, the difference between the two representing the total runtime. Because the $G_x^2 + G_y^2$ calculation requires several convolution calculations, which in turn could be the assembly of several smaller tiles, the run times of each circuit are also summed for the final overall quantum run time.

```

1  start_time = time.time()
2  backend.run(circuit).result()
3  elapsed_time = time.time() - start_time

```

Listing 3.1: Pseudocode of the runtime calculation.

Circuit shape and memory requirements

Qiskit has built functions to calculate performance metrics related to circuit shape. Circuit depth can be found using qiskit's *depth()* function, while circuit width, which represents the amount of qubits used in a circuit, is found with *width()*. Information about the number of total gates is found with *count_ops()*, with this project grouping them into one-qubit gates and two-qubit gates. One-qubit gates used in the circuits were Ry, H, Measurement, Rz, and X gates. Two-qubit gates used in the circuits were CX and CZ gates. It should be noted that if these circuits were to be transpiled to ensure compatibility with real QPUs, the types of gates would likely differ.

Regarding image shape, the implementation of QCrank used for this paper requires that the number of pixels used to input images be powers of 2. For simplicity, this project used square images with lengths that are powers of 2. Many experiments used several QCrank compatible image sizes. Due to the experiments being run on a simulator, image size needed to be regulated to ensure the generated circuits basis states all fit in memory. The primary cause for memory issues, storing a circuit's basis state information, scales with the circuit width. N qubits required 2^N basis states, with each basis state represented by a 16B complex double precision floating point number when represented classically[8]. Given that the maximum memory capacity of the GPU nodes on Perlmutter is $4 \times 40 = 160\text{GB}$ distributed among the 4 GPUs, 10 billion basis states can be stored at a time on the GPUs alone. The CPU that controls the GPU nodes provides an additional 256GB of memory,

giving a total of 416GB of memory for 26 billion basis states. Given this upper limit for basis state storage, a maximum of 34 qubits can be used, as a circuit of that width has about $2^{34} = 17$ billion basis states. However, to reduce the CPU's impact on GPU simulation and better compare it to purely CPU circuit simulation, the 256GB of memory supplied by the CPU in the GPU node will not be considered for memory available for the circuit. As a result, 33 qubits is the upper limit used in simulations conducted on GPU nodes. Perlmutter CPU nodes have a larger total memory space of 512GB, allowing them to fit 32 billion basis states. The CPU nodes' larger memory capacity provides a small advantage in the maximum circuit with 34 qubits, but given the poorer performance of the CPU exclusive simulations[8] circuits of that size were not tested.

It should also be noted that fitting within each node's memory does not mean that the circuit will run within a reasonable amount of time. The number of shots scales quadratically with the length of the image (or tile), further increasing the runtime as the image size increases. In order to keep execution of experiments within the allocated maximum session time for each Perlmutter node, the experiments of this thesis only used circuit sizes well below the maximum memory requirement.

Fig. 3.1 shows what image and circuit combinations can fit within the CPU and GPU nodes. The G_x^2 and G_y^2 calculations for the extended circuits incur a high base memory cost, requiring $2 * (6 + 6 + 4) = 32$ data and ancilla qubits. Squared stencil calculations using the extended circuit implementation are consequently unable to be simulated using only GPU memory even at the smallest image sizes, as an 8x8 image is the smallest QCrank compliant

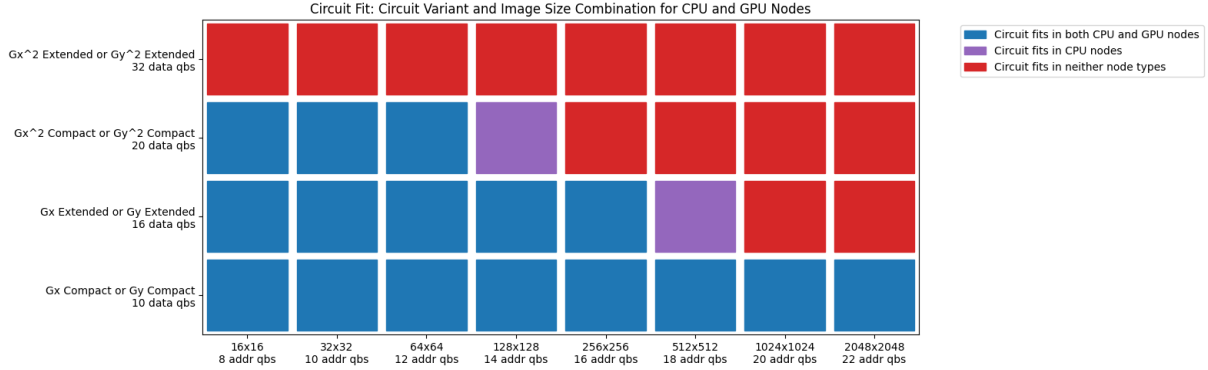


Figure 3.1: This chart displays which combinations of circuit variants and image sizes fit into the Perlmutter CPU and GPU nodes. Combinations that produce a circuit width of 34 are the only ones that can fit in a CPU node but not in the GPU memory of the GPU nodes.

image that has some Sobel values with fully valid neighbors. The compact circuit's minimum memory requirements is better, requiring $2 * (6 + 4) = 20$ data and ancilla qubits. Given the square inputs used in the experiments, the number of address qubits will always be even. This leads to a maximum $33 - 20 = 13 - (13\%2) = 12$ address qubits, which translates to a maximum image size of $2^6 \times 2^6 = 64 \times 64$ that can be computed for this combination of circuit variants and hardware.

Circuit Reliability

In order to check the functionality of all the circuits that can be simulated, the first experiment compares the circuit results with ground-truth values that were calculated with purely classical operations. For the EHands encoding and arithmetic sub-circuits this experiment

compared each circuit's output with its expected results 100 times, while for the larger G_x , G_y , G_x^2 , and G_y^2 circuits the experiment ran each circuit with a 32×32 image populated with random values, and compared the output with its fully classically calculated counterpart. Because each 32×32 image produce about 1000 values, this was only done once per stencil or squared stencil calculation circuit. To analyze the results, the experiment created two types of graphs, with an example of each seen in Fig. 3.2. The first type analyzed residuals, which were the difference between the ground-truth classical results and the quantum circuit results. The second type was a scatter plot that the classical results against the quantum circuit results.

For the residuals graph, the mean of the residual values should be roughly around 0. As mentioned in Sec. 2.2, the standard deviation values for the circuits in an ideal simulation should follow the formula $\sigma = N_s^{-\frac{1}{2}}$, where N_s is the number of shots per address[4]. All circuits used 5000 shots per EHands calculation and 5000 shots per address for the Sobel calculations, which should yield standard deviation values around 0.014. However, as mentioned during the implementation sections for the extended and compact circuits, because of the weighted summations needed for the stencil calculations, the probability amplitudes of the expected values are scaled down.

To obtain the expected standard deviation value, if the results need to be scaled by a factor of s to achieve their intended value, then the number of shots would need to be scaled by s^2 [3]. This would have adverse affects on runtime, especially for larger circuits that already have a high total number of shots due to the scaling with number of addresses.

Given a limited session time to simulate circuits on Perlmutter nodes, runtimes needed to be kept to a minimum. Although not ideal, the comparison values were scaled down by the same magnitude instead. For extended circuits, the scaling factor would be 6 for the stencil operation circuits and 36 for the squared stencil operation circuits. For compact circuits, the scaling factor would be 4 for stencil operations and 16 for the squared stencil operations. Alternatively, the expected standard deviation for the different circuit variants could be calculated separately, but having one constant standard deviation value was deemed simpler during testing.

For the classic vs. quantum-augmented Sobel graphs, a best-fit line was created using numpy's `polyfit()` with `degree=1`. Its angle θ was calculated with the formula $\theta = \frac{\arctan(m)*180}{\pi}$ where m is the slope coefficient of the best-fit line. Finally, the correlation coefficient was calculated using numpy's `corrcoef()` function. Ideally, the best-fit line should be $y = x + 0$, with a theta of 45 and a coefficient value of 1. These imply the values of the classical and quantum circuit results are all close to each other, with no bias.

Experiment Design

Experiment 1 is a verification check of the core EHands arithmetic computations, as well as the quantum circuits used to calculate the Sobel stencil computations. The experiment involves testing 100 random inputs for each EHands sub-circuit and testing each Sobel circuit that can fit in memory on a randomly generated 32×32 image. Specifically, only the three

combinations of circuit variant and calculations in Fig. 3.1 that can run 32×32 images will be tested. Quantum-augmented results, their fully classical ground truths, and the residuals between the two sets will be used to generate Classical vs. Sobel-Augmented value graphs and Residual Histograms. To visually verify its effectiveness, each circuit will also be run on a real picture, with its output compared to the input and fully classical Sobel comparison.

Experiment 2 focuses on comparing the compact and extended circuit variants of the Sobel Stencil calculation. The G_x circuit of both circuit variants will be applied to images of sizes 16×16 , 32×32 , 64×64 , 128×128 . This experiment generated a chart comparing both implementations' runtimes, as well as a table to analyze their circuit shapes and number of 1 and 2 qubit gates. Based on the results of this experiment, the subsequent experiments will use the variant that is better based on the performance metrics analyzed here.

Experiment 3 focuses on comparing the runtime of the Perlmutter CPU nodes with its GPU nodes. For both nodes, the full Sobel implementation will be conducted by G_x and G_y calculations using the compact Sobel circuit. The total runtime will be compared on both node types for input images of sizes 16×16 , 32×32 , 64×64 , and 128×128 in a runtime plot. For the largest image, the quantum-augmented results of the full Sobel results will be compared with the original input image, as well as with a fully classically computed result.

Experiment 4 demonstrates the effectiveness of tiling as problem size grows by comparing an implementation with no tiling to an implementation that makes use of tiles one-quarter the size of the original image with overlap. These implementations will calculate G_x for images sizes of 16×16 , 32×32 , 64×64 , 128×128 , and 256×256 . The runtime of the

single circuit implementation will be compared with the serial runtime of the tiled circuit via a chart. Additionally, a table that compares the single circuit runtime to the theoretical parallel runtime for each image will be created. The theoretical runtime assumes all tiles can be run simultaneously on different QPUs, with the slowest runtime value among the tiles taken for each image size.

Experiment 5 plots the change in runtime as number of tiles increases for a 256×256 image, displayed via a plot of tile count vs. runtime. The tile lengths tested for this experiment were 16, 32, 64, and 128.

3.3 Experiment 1: Verification of Correctness of Quantum Arithmetic Computations

This section examines the accuracy of the EHands circuits, as well as the Sobel circuits that could fit within memory within Perlmutter’s CPU and GPU nodes. The comparison values generated by the fully classical calculations are scaled down to the expected values of the quantum outputs, as discussed in Sec. 3.2. This experiment generated charts for the fully classical vs. quantum-augmented results of EHands EVEN encoding and decoding, EHands Arithmetic, compact and extended G_x and G_y , and compact G_x^2 and G_y^2 . For conciseness, only charts for extended G_x will be shown here in Fig.3.2, with the rest of the charts found in the Appendix.

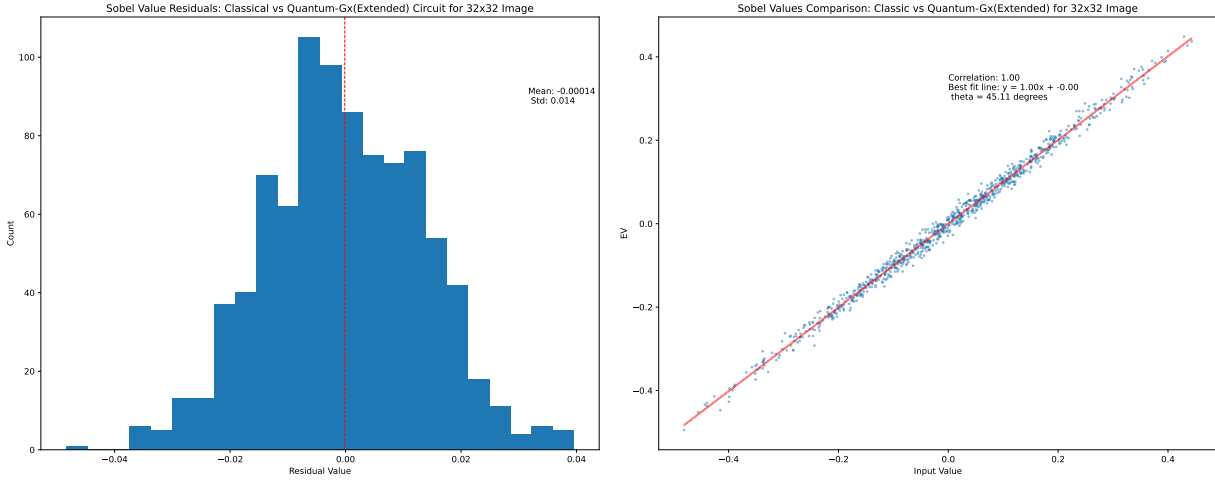


Figure 3.2: Two graphs used to verify the correctness of computations of the extended quantum-augmented Sobel circuit when calculating G_x by testing it on a randomly generated 32×32 image and comparing it to scaled down fully classically calculated values. Both produce expected results, suggesting the circuit is working as intended.

Both figures suggest that the circuit is working as intended. The correlation of 1.00 and best-fit line angle of about 45° in the value comparison chart suggest that the values produced by the circuit were close to expected, with the line passing through the origin point suggesting that there is no unintended bias in the results. The residual histogram further supports that these circuits work correctly, with a mean close to 0 suggesting that there is no issue that causes an offset in expected calculations and standard deviation values close to the expected value of 0.014.

While most of the sub-circuits tested here have their quantum-augmented results compared directly to their scaled down classical equivalent, test results for verifying the effectiveness of the parity flip operation needed to be handled indirectly. Because it is difficult to test

Circuit	Res. Mean	Res. Stdev.	Corr.	Best-fit line	Angle
Encoding	0.0014	0.013	1.00	$y=1.00x+0$	45.03
Multiplication	0.0012	0.014	1.00	$y=1.00x+0$	45.0
Weighted Sum	-0.00049	0.013	1.00	$y=1.00x+0$	45.0
Reversible Parity	0.00094	0.014	1.00	$y=0.99x+0$	44.74
Old Parity	-0.001	0.014	1.00	$y=1.00x-0$	45.11
Compact G_x	-0.00011	0.013	1.00	$y=1.00x-0$	44.94
Compact G_y	0.00078	0.014	1.00	$y=1.00x+0$	44.94
Extended G_x	-0.00014	0.014	1.00	$y=1.00x-0$	45.11
Extended G_y	-0.00037	0.013	1.00	$y=1.00x-0$	45.02
Compact G_x^2	-0.00037	0.014	0.99	$y=1.00x-0$	44.95
Compact G_y^2	-0.00025	0.014	0.99	$y=1.00x-0$	44.95

Table 3.1: Summary of key values in the 20 output analysis charts created during Experiment 1. The two parity circuits were tested via a sequence of weight summation blocks. All produce intended results. All charts used to build this table can be found in the Appendix.

the parity flip in isolation as its primary purpose is to clean up the quantum state after an EHands weighted summation block, its tests analyzed the accuracy of a sequence of weighted summation, which would provide inaccurate results if the parity flip block implementation was not working as intended.

Table 3.2 summarizes the important values of the graphs for this experiment. It contains the mean and standard deviation values when taking the residual values between the fully classical and quantum-augmented Sobel circuits and EHands sub-circuits. Additionally, it provides information from the scaled down classical vs. quantum circuit results plot. From the table it is clear that all quantum sub-circuits are producing expected results. All residual means are close to 0, and all residual standard deviations are around the expected

value of 0.014. The quantum-augmented vs. scaled down comparison values also imply the circuits are functioning correctly, with the best-fit lines of each plot running through the origin and the about 1.00 correlation suggesting the values are close to expected.

3.4 Experiment 2: Comparison of Extended and Compact Circuit Variants

This section compares the runtime, circuit shape, and total number of 1-qubit and 2-qubit gates of the extended and compact circuit variants when calculating G_x . The goal of this experiment is to decide which of the variants is superior regarding these metrics. The analysis of the runtime will provide insight on which is more efficient in simulation environments. Gate count and circuit depth, while also important for analyzing a circuit's efficiency during simulation, have a more important role when using real QPUs. Errors caused by hardware problems in real QPUs encourage the use of smaller circuits to reduce the amount of compounding errors.

Fig. 3.3 compares the runtime of both circuit variants when calculating G_x at image sizes provided in Sec. 3.2. The compact circuit conducted its simulated runs much faster at all sizes because of its smaller state vector space.

Table 3.2 compares the circuit shape and gate counts of the compact and extended circuits when calculating the G_x stencil for the images used in this experiment. Although

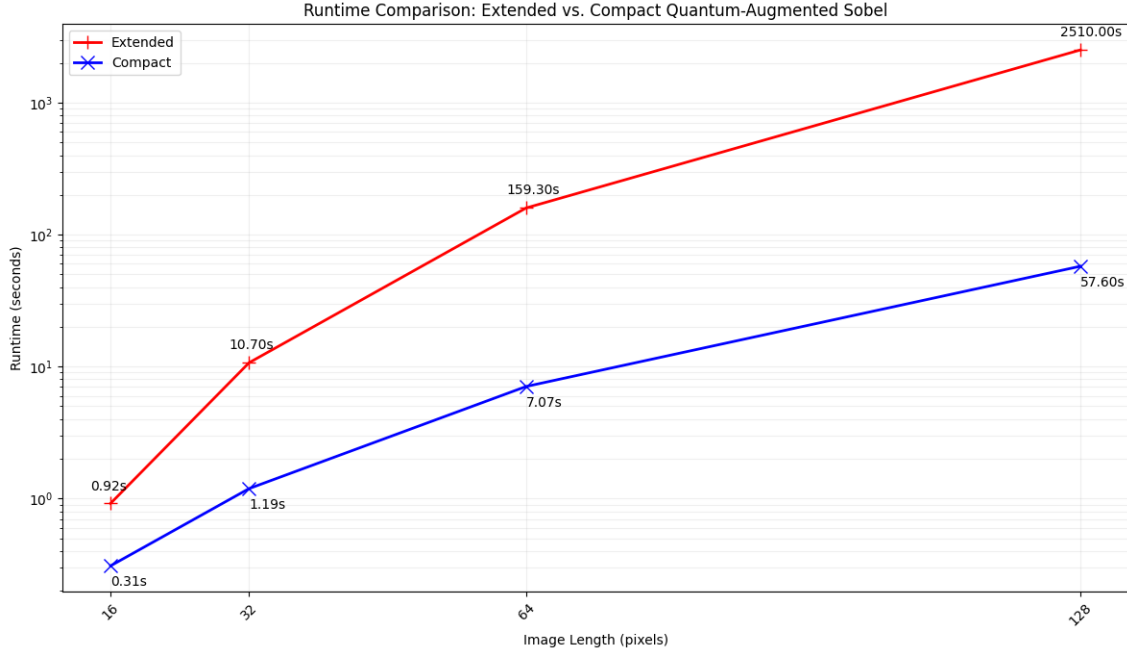


Figure 3.3: A comparison of runtimes between the compact circuit implementation and the extended implementation when calculating G_x at various image sizes. Note log scaling on the y-axis. The compact circuit is faster at all image sizes

the compact circuit has smaller values for each metric, the difference is small relative to the values. This is mainly because the QCrank encoding sub-circuit, which takes up most of the total circuit depth, is identical between both variants. Differences between the two variants come the EHands portion of the circuits, both of which are of constant shape and gate count no matter the size of the image. The main point of improvement is found in circuit width. A 6 qubit difference between the circuit variants means that the extended circuit has $2^6 = 64\times$ more basis states, which incurs a heavy impact in simulated runtime at larger circuit sizes.

Analysis regarding the performance of the compact and extended circuit variants will be

	16x16	32x32	64x64	128x128
Compact Width	18	20	22	24
Extended Width	24	26	28	30
Compact Depth	534	2070	8214	32790
Extended Depth	541	2077	8221	32797
Compact 1qb gates	1575	6187	24623	98355
Extended 1qb gates	1587	6199	24635	98367
Compact 2qb gates	1550	6158	24590	98318
Extended 2qb gates	1556	6164	24596	98324

Table 3.2: A comparison of circuit shape and number of 1 and 2 qubit gates between the compact and extended circuit variants for various image sizes. The circuits have similar shapes and gate counts, but any difference in circuit width is impactful at larger values.

covered in more detail in Sec. 3.8, but from the better runtime scaling with image size and slightly more efficient circuit architecture, it is clear that the compact variant is superior. As a result, all of the following experiments will use these variants in their simulations.

The compact implementation could not compare its G_x^2 and G_y^2 computation circuit with the extended implementation due to the latter's minimum circuit size exceeding node memory. However, the runtime of the compact implementation calculating G_x^2 on a 32×32 image was also recorded for comparison with the nonsquared calculation circuits. It took about 304 seconds to run because of its increased circuit width, while the G_x stencil computation circuits only took slightly longer than 1 second to run at the same image size. Due to the poor scaling of the compact squared stencil circuits regarding runtime, the squaring calculations are best done classically for simulations.

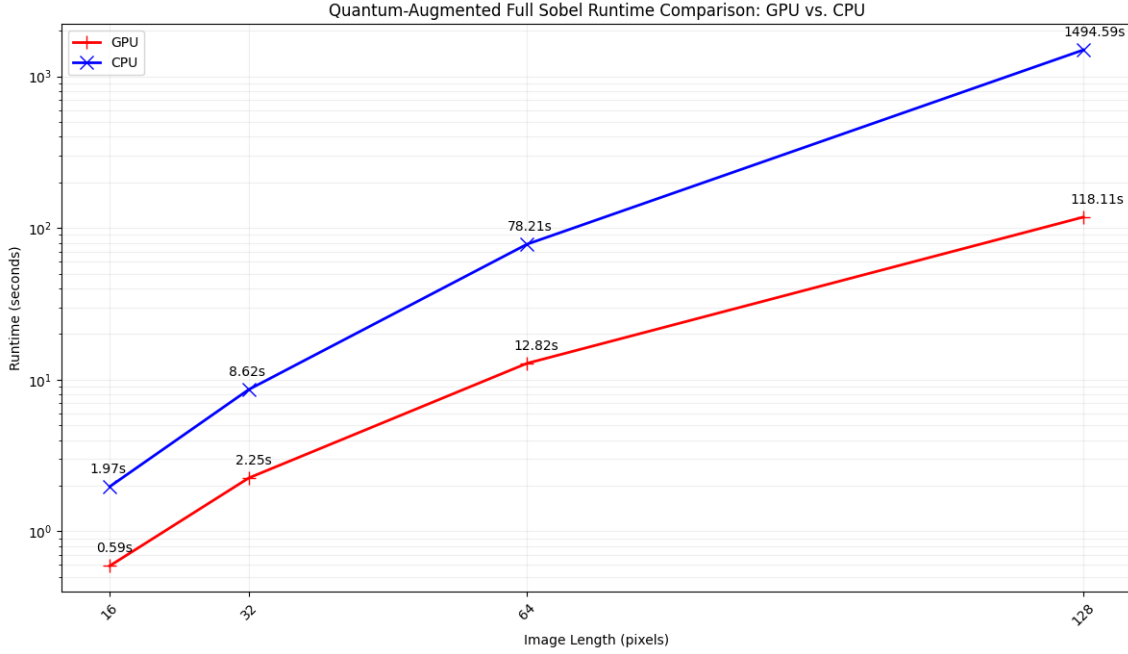


Figure 3.4: A comparison of runtimes of the G_x calculation when ran on a CPU node vs a GPU node for various image sizes.

3.5 Experiment 3: CPU vs. GPU Runtime

Comparison

This section compares the runtime of the compact quantum-augmented Sobel implementation by running the full Sobel gradient magnitude calculation $G = \sqrt{G_x^2 + G_y^2}$ for identical image sizes on both types of Perlmutter nodes, as defined in Sec. 3.2. To perform the complete Sobel calculation, separate serially executed compact Stencil operation circuits were used to calculate G_x and G_y , with the remaining calculations handled classically.

Fig.3.4 shows the runtimes measured for this experiment. GPU-assisted simulation out-

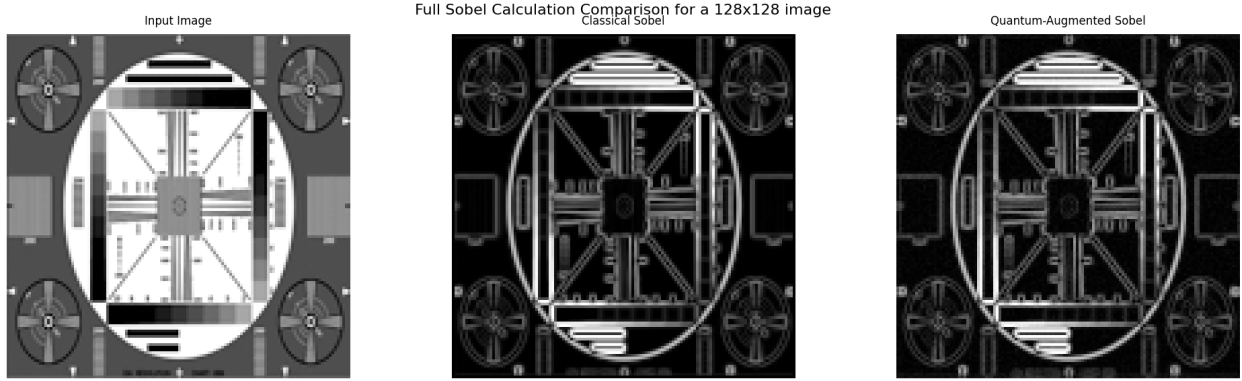


Figure 3.5: A comparison of the original 128×128 image, the fully classical Sobel comparison, and the Quantum-Augmented Sobel output.

performed pure CPU simulation in terms of runtime at all problem sizes, with greater effectiveness as image size increased. For 16×16 images the GPU node ran its experiment about 3x faster, and was about 12x faster for the 128×128 image calculation.

Fig.3.5 provides a comparison between the largest input image, its fully classical Sobel output, and the quantum-augmented results produced from this experiment as an additional way to show the implementation's effectiveness. Visually, the comparison and quantum-augmented outputs are very similar. Both Sobel filter images recognize the same edges with visually similar gradient magnitude values. However, close inspection of solid regions of color in the quantum-augmented output shows some minor changes in color values caused by sampling error. Increasing the number of shots per address will likely reduce these slight errors in value, as it reduces the standard deviation of values produced by the circuit.

3.6 Experiment 4: Comparison between Tiling and Increasing Circuit Size

Fig.3.6 compares the runtimes between the calculation G_x on the entire image and the calculation G_x carried out on quadrant-sized tiles that were reassembled into a final output image. The image sizes used are outlined in Sec. 3.2. Due to the use of overlap to ensure that the invalid tile borders would not be needed in image reconstruction, 9 tiles are used instead of 4, following the formula $(m + 1)^2$, where m is the number of tiles required to fill out the output image without overlap, which was first used in the tiling subsection of Sec 2.1.

The third line in the figure represents the theoretical runtime of the tiling implementation if all circuits were run in parallel, with the value taken from the slowest runtime among the tiles. An additional assumption for these hypothetical values to be valid is that there are enough GPU's available to run each in tile at the same time. Execution of tiling in parallel both on GPUs and QPUs is the subject of potential future work.

From the two real lines, the tiling and full image calculations have shorter runtimes at different image sizes. The lack of overhead and smaller changes in runtime as qubit count increases for smaller quantum circuits allowed the full image approach to outperform the serially executed smaller tile circuits, with the tiling approach taking about the 3x longer for the quantum simulations to execute. However, as the growth in the number of basis states became more detrimental to the simulation speed for larger image sizes, the tiling method started to match the full image implementation with regards to simulation runtime for the

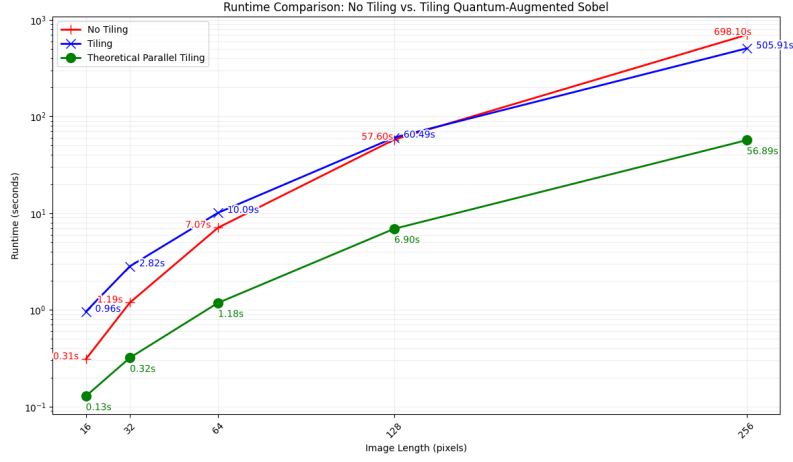


Figure 3.6: A comparison of runtimes between no tiling, tiling with quadrant sized tiles, and the theoretical runtimes if the tiles were calculated in parallel. The serial tiling execution only starts to outperform the implementation that uses the full image at the 256×256 image.

	16x16	32x32	64x64	128x128	256x256
No Tile Width	18	20	22	24	26
Tiled Width	16	18	20	22	24
No Tile Depth	534	2070	8214	32790	131094
Tiled Depth	150	534	2070	8214	32790
No Tile 1qb gates	1575	6187	24623	98355	393271
Tiled 1qb gates	419	1575	6187	24623	98355
No Tile 2qb gates	1550	6158	24590	98318	393230
Tiled 2qb gates	398	1550	6158	24590	98318

Table 3.3: A comparison of circuit shape and number of 1 and 2 qubit gates between the compact and extended circuit variants for various image sizes.

128×128 image and outperform it by 40% for the 256×256 image. If hardware permits parallel tiling, the tiling method would outperform at all problem sizes, provided that the theoretical behavior is correct.

Table 3.3 shows a comparison between the circuit shape and the number of gates between the two implementations to help explain the effectiveness of the tiling method. While the no tile and tiling methods differ by only 2 qubits for all image sizes, adding two qubits quadruples the amount of memory used, which becomes more of a detriment as overall circuit size increases. The circuit depth for the tiled version is also about 3.5 to 4 times higher than the non-tiled depth for the tested image sizes. The difference in circuit shape, alongside the increased shot count values due to their scaling with number of pixel values, further increased runtime issues.

3.7 Experiment 5: Comparison of Tiling Sizes for a Fixed Size Image

This experiment examine how the tile length, and by extension the number of tiles, affects the runtime of a compact quantum-augmented G_x calculation for a 256×256 image. Theoretical parallelization was not utilized here, as some of the smaller tile length values would require hundreds of tiles to run fully in parallel, which is unrealistic with current hardware.

It should also be noted that for the simulation that used tile lengths of 16, three total extra tiles were needed for overlapping, as with only one tile's worth of extra length there would not be enough overlap without using tile border regions, as discussed in Tiling subsection

Tile Length	Tiles needed per dimension if no overlap is used	Number of overlapping pixels per tile in 1D with 1 extra tile
16	16	1
32	8	4
64	4	16
128	2	64

Table 3.4: Shows the relationship between tile length and how many tiles are needed to fill out one dimension of the 256×256 image if no overlapping is used. The third column calculates the amount of overlapping addresses are allotted to each tile in one dimension of the image.

of Sec. 2.1. Table 3.4 helps visualize this problem. As the tile length decreases, more tiles would be needed to fill one dimension of the output image, and the extra length used for overlap is spread out across more tiles. When the length per tile equals the number of tiles needed to fill one direction of the output image without overlap, each tile is only assigned 1 pixel of overlap. As mentioned in Tiling Stitching subsection of Sec. 2.1, rather than using invalid tile border values, these addresses are skipped, producing seams in the output image.

Adding 2 extra tiles per dimension was a temporary fix, and a more robust solution to this issue is planned as future work. Because this update changes the behavior of tile count growth from $(m+1)^2$ to $(m+3)^2$, where m is the number of tiles needed to fill one dimension of the output image without overlap, the results of this tile length produced results that did not follow the pattern of the other 3 tile lengths tested, seen in Fig. 3.7. The runtime data point for the implementation that used tiles of length 16 was included to show the impact of adjusting the amount of extra tiles affect simulation runtime.

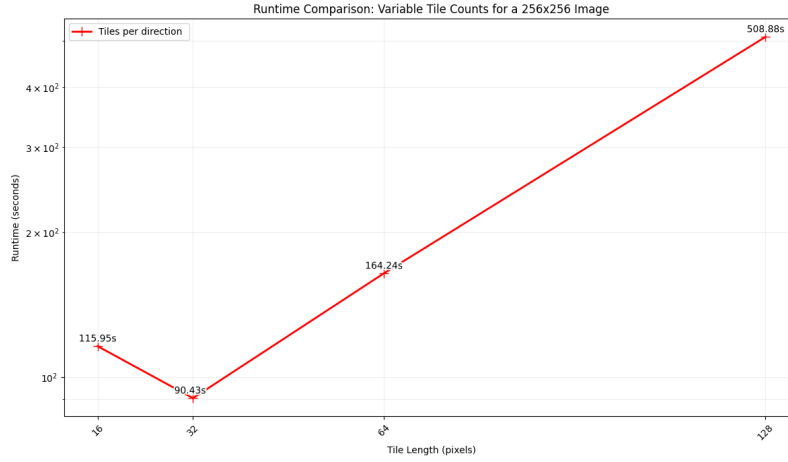


Figure 3.7: A graph depicting how runtime of the G_x computation on a 256×256 image changes with length of tiles. Tile length 16, which needed 3 extra tiles per image dimension instead of 1, produces a runtime that does not follow the pattern of the other tile length. While these other values produce a straight line on this figure, note that logarithmic scaling of y-axis, which instead suggests an exponential scaling of runtime.

Fig. 3.7, plotting the runtime values for tile lengths of 32, 64, and 128 produce a nearly straight line with logarithmic scaling of the y-axis. This implies an exponential relationship between runtime and tile length, with the caveat that the tile length never equals or exceeds the number of tiles used in one dimension.

Table 3.5 shows a breakdown of the amount of tiles used for each tile length, as well as the total runtime and runtime per tile. As stated previously, tiles of length 32, 64, and 128 required $(m + 1)^2$ tiles, while tiles of length 16 require $(m + 3)^2$ tiles because with only one extra tile the invalid border regions of each tile would need to be used. From this table it is clear that the simulation where tile length was 16 pixels had a smallest runtime per tile, but the two additional overlapping tiles in each direction compared to the other simulations

Tile Length	Total Tiles	Total Serial Runtime (s)	Runtime Per Tile(s)
16	361	116.0	0.32
32	81	90.4	1.11
64	25	164.2	6.56
128	9	508.9	56.5

Table 3.5: Summary of the amount of tiles used when each tile is a given length, as well as the overall runtime and the average runtime per tile length. Tile length 16 has the smallest runtime per tile, but has a higher total serial runtime compared to tile length 16 because more extra tiles are required in each dimension of the image

were enough to make the overall runtime slower than the simulation that used tiles of length 32.

3.8 Findings and Discussion

Analysis of Experiments

Experiment 1's primary role was verification that the implementation works as intended before moving on to the more involved experiments that follow it. This is especially important for the new version of the parity flip gate block, which was developed in this thesis and was thus not vetted by previous work. Likewise, the EHands-QCrank circuits for the quantum-augmented Sobel stencil operations are original designs, and thus also needed to be thoroughly tested before they could be confidently used in the experiments that followed.

With regard to Experiment 2, while the difference of 6 qubits between the compact

and extended designs is not very impactful at smaller image sizes, the exponential scaling of basis states with circuit width means that the extended circuit variant will suffer from runtime increases much sooner and much harder compared to the compact implementation. This experiment was important in showing that the compact circuit implementation is more useful for quantum simulations. Its superiority over the extended circuit variant should also extend into runs on real quantum hardware, as the decrease in circuit width should also reduce the total amount of hardware related errors in the system.

Experiment 3's focused entirely on examining simulation performance. Although previous papers have observed that CPU simulation outperforms GPU-accelerated simulations at low circuit sizes due to the overhead cost that overshadows gains in parallelism [8], this is not the case with the experiments conducted for this paper. This is likely due to the high minimum circuit width for calculations, even with the compact circuit implementation. The knowledge that simulations on Perlmutter CPU nodes will never outperform GPU nodes for the provided circuits removes the burden of choice regarding which node to use. The sheer difference in runtime performance for the largest image size between the two node types further highlights the effectiveness of GPU over CPUs for experiments with large circuit widths and shot counts.

Experiment 4 provides information on the strengths and weaknesses of the divide-and-conquer tiling strategy compared to handling the entire picture in its entirety. If parallel execution of tiles is not an option due to lack of hardware resources, tiling performs worse compared to computation on full images for small circuit sizes. The weaker runtime per-

formance of the tiling implementation on small images does not even take into account the overhead of creating tiles and reassembling the final image. With small circuits, the runtime increases of simulating the extra tiles needed for overlap overshadow the decreases that come with decreasing circuit width. However, at larger circuit widths when adding qubits have a much greater impact on simulation runtime, simulating extra tiles is more desirable. Parallel executions, which were not simulated in this experiment, should offer better runtimes at all image sizes, provided it follows the behavior of the hypothetical runtimes that come from the slowest runtimes of each group of tiles. While the runtime behavior learned from this experiment’s simulations shouldn’t be used to predict runtime behavior on real QPUs, tiling still has potential use when running experiments on real QPUs. The lower gate counts and circuit sizes should reduce the amount of compounding hardware errors in QPUs, as discussed in Experiments 2 and 4. Future work comparing tiling to full-image quantum-augmented Sobel calculations using real QPUs is needed to verify this assumption.

Experiment 5 attempts to find which combination of tile length and tile count best improves execution time. Due to constraints with maximum simulation time on Perlmutter, a set of relatively small images were used. As a result, only a small range of tile length values could be tested before the number of extra tiles per dimension needed to be increased. Despite this, a clear pattern of exponential increases in runtime is observed when examining the tile length values that make use of only 1 extra tile per dimension. From the limited problem size, it appears that the combinations of tile length and number of tiles just before the need for more extra tiles provide the best runtime in a simulated environment. However,

for larger image sizes this could result in hundreds or thousands of separate circuits to simulate all the tiles needed, with the 256×256 image used in this experiment already needing 81 tiles for its best tile length and tile count combination.

The first key takeaway from these experiments is that the quantum-augmented Sobel implementation can produce results comparable to purely classical Sobel outputs, at least when using ideal simulators. Experiment 1 provides confidence that each block is implemented correctly, especially crucial for the novel parity flip implementation and the stencil calculation circuits. The new parity flip variant’s primary improvement of reversibility was not utilized to its full extent in this paper, but it did allow for much faster simulation because of the removal of mid-circuit measurements. The reduced simulation runtimes allowed for testing on much larger problem sizes. Experiment 3 shows that the bulk of the Sobel operations, the computation of G_x and G_y , can be generated in a quantum environment and help generate an output almost identical to a fully classical implementation. Experiments 4 and 5 highlights the power of tiling when leveraging quantum computing on larger image sizes.

Limitations with Simulations

As mentioned in the previous subsection, due to the fundamental differences between simulations on classical hardware and real experiments with QPUs, there will be differences when the quantum-augmented circuits are implemented on real quantum hardware. The

runtime behavior of simulations would not be the same as real quantum circuits, so runtime analysis conducted between the compact and extended circuits, as well as the various tile sizes would not be applicable on real hardware. The memory limitations caused by circuit width are also issues that exist solely for simulated circuit runs, but real QPUs do have a related issue with circuit shape. The circuits simulated in this paper would need to be transpiled to match the gate composition and qubit mapping of the real hardware.

3.9 Potential Future work

Due to hardware and time limitations at the time of running these experiments, there are several obvious next steps to expand on their results.

Runs on Real QPUs

Access to real QPUs was not available for the circuits used in this paper during development. Work is currently being done to gain access to real QPUs via IBMQuantum. With access to IBMQ's QPUs, larger circuits and image sizes would be able to be tested with less worry about runtime or memory. The larger circuit variants and calculations which could not fit into memory for simulations, such as the full $G_x^2 + G_y^2$ calculation, could be tested. The impact of quantum errors as the size of the circuit increases can be tested. Slightly more compact circuits that make use of a single resetting ancilla qubit for all parity flip gates can also be tested. The function of the stencil calculation circuits do not require reversibility,

but reversibility was needed to utilize the state vector simulator that allowed for simulation runtimes to fit within session time limits at Perlmutter.

The impact of tiling on reducing quantum errors in the Sobel calculation is another experiment that could not be covered with ideal simulations. Comparison of changes of runtime, if any, for parallel execution of tiles circuits and a circuit with the full image could also be conducted, as real QPU runtimes should behave differently compared to simulations. For parallel executions, QCrank has built-in functionality for creating circuits for parallel runs in a batch job, but this feature was not utilized in this project, as serial runtime was deemed more important to analyze first. It should be noted that IBMQ enforces a maximum shot count per job, so there is still a limit to the circuit and image sizes that can be realized if the same level of precision used in the experiments is maintained.

Increasing viable range for tile length

For Experiment 5, a temporary fix was needed when the number of extra tiles needed to be increased for the smallest tile length. The tiling implementation in this thesis worked around this by adding two additional tiles per dimension for this tile length. A more robust implementation of the tile creation to allow for smaller tile lengths is planned, but given the worsening runtime performance with extra overlapping tiles, the ideal tile length per image appears to be before this change is needed, anyway. This behavior might change for extremely large image sizes, but the runtime gains would mainly be in simulations. This

is because to hit this threshold hundreds or thousands of tiles would be needed and IBMQ enforces a cap on the number of circuits that can be run for a QPU. As a result, there may not be a use case for this feature.

Value averaging for addresses covered by multiple tiles

For the current implementation of the output image construction when using overlapped tiles, if an address is covered by multiple tiles, the older values are replaced when another tile updates the value of the address. A potential non-quantum optimization would be to instead take the average of each values associated with each address. This could further lower the error for regions of overlap in the final image.

Chapter 4

Conclusion

This paper provides a novel way to calculate the Sobel Filter by leveraging quantum computing, specifically by expanding on the foundation of QSobel with LBNL’s QCrank and EHands protocols for more efficient encoding and quantum arithmetic. An important improvement developed during the creation of this paper’s codebase is a new EHands parity flip block which removes the need for mid-circuit measurements, making the full EHands protocol reversible. This reversibility allows for the use of the time-efficient statevector simulator, which reduces runtime for the experiments in this thesis. More importantly, reversibility for all of EHands’ circuit blocks expands the list of algorithms it is compatible with. After the verification of function for the EHands blocks and individual gradient circuits implementation, this project highlighted their effectiveness in accurately reproducing Sobel Filter outputs by substituting the Stencil operations with quantum circuits. Despite limitations caused by classical memory, these circuits were able to simulate gradient calculations for

images up to 256×256 . The divide and conquer strategy of tiling provided another means of optimization, which provides better simulation runtime performance and potentially lower quantum errors on real quantum hardware. While untested on real quantum hardware, the groundwork has been laid through confirmation that the circuits work in simulations. If future experiments on QPUs produce positive results with current NISQ hardware, this method can prove to be a viable alternative to classical Sobel calculation, and the foundation for work on other quantum-augmented image processing algorithms.

Bibliography

- [1] Mercy G Amankwah et al. “Quantum pixel representations and compression for N-dimensional images”. In: *Scientific reports* 12.1 (2022), p. 7712.
- [2] Jan Balewski. Personal Communications. Apr. 2025.
- [3] Jan Balewski et al. “EHands: Quantum Protocol for Polynomial Computation on Real-Valued Encoded States”. In: *arXiv preprint arXiv:2502.15928* (2025).
- [4] Jan Balewski et al. “Quantum-parallel vectorized data encodings and computations on trapped-ion and transmon QPUs”. In: *Scientific Reports* 14.1 (2024), p. 3435.
- [5] Andrew J Daley et al. “Practical quantum advantage in quantum simulation”. In: *Nature* 607.7920 (2022), pp. 667–676.
- [6] Hana Ben Fredj et al. “Parallel implementation of Sobel filter using CUDA”. In: *2017 International Conference on Control, Automation and Diagnosis (ICCAD)*. IEEE. 2017, pp. 209–212.

- [7] Lov K Grover. “A fast quantum mechanical algorithm for database search”. In: *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*. 1996, pp. 212–219.
- [8] Hiroshi Horii, Christopher Wood, et al. “Efficient techniques to gpu accelerations of multi-shot quantum computing simulations”. In: *arXiv preprint arXiv:2308.03399* (2023).
- [9] IBM. *Aer Simulator Documentation*. Qiskit Documentation. <https://qiskit.github.io/qiskit-aer/index.html>. URL: <https://qiskit.github.io/qiskit-aer/index.html>.
- [10] Bernd Jähne. *Digital image processing*. Springer Science & Business Media, 2005.
- [11] Jeremy Johnson and Markus Puschel. “In search of the optimal Walsh-Hadamard transform”. In: *2000 IEEE International Conference on Acoustics, Speech, and Signal Processing. Proceedings (Cat. No. 00CH37100)*. Vol. 6. IEEE. 2000, pp. 3347–3350.
- [12] Phuc Q Le, Fangyan Dong, and Kaoru Hirota. “A flexible representation of quantum images for polynomial preparation, image compression, and processing operations”. In: *Quantum Information Processing* 10 (2011), pp. 63–84.
- [13] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Cambridge university press, 2010.
- [14] Minati Rath and Hema Date. “Quantum data encoding: A comparative analysis of classical-to-quantum mapping techniques and their impact on machine learning accuracy”. In: *EPJ Quantum Technology* 11.1 (2024), p. 72.

- [15] Maria Schuld and Francesco Petruccione. *Machine learning with quantum computers*. Vol. 676. Springer, 2021.
- [16] Irwin Sobel et al. “Sobel-feldman operator”. In: *Preprint at [https://www. researchgate. net/profile/Irwin-Sobel/publication/285159837](https://www.researchgate.net/profile/Irwin-Sobel/publication/285159837)*. Accessed 20 (2022), p. 30.
- [17] Bo Sun et al. “An RGB multi-channel representation for images on quantum computers”. In: *J. Adv. Comput. Intell. Intell. Inform* 17.3 (2013).
- [18] Fei Yan and Salvador E Venegas-Andraca. “Lessons from twenty years of quantum image processing”. In: *ACM Transactions on Quantum Computing* 6.1 (2025), pp. 1–29.
- [19] Xi-Wei Yao et al. “Quantum image processing and its application to edge detection: theory and experiment”. In: *Physical Review X* 7.3 (2017), p. 031041.
- [20] Rahat Yasir, Md Ashiqur Rahman, and Nova Ahmed. “Dermatological disease detection using image processing and artificial neural network”. In: *8th international conference on electrical and computer engineering*. IEEE. 2014, pp. 687–690.
- [21] Yi Zhang, Kai Lu, and YingHui Gao. “QSobel: A novel quantum image edge extraction algorithm”. In: *Science China Information Sciences* 58 (2015), pp. 1–13.
- [22] Yi Zhang et al. “NEQR: a novel enhanced quantum representation of digital images”. In: *Quantum information processing* 12 (2013), pp. 2833–2860.

- [23] Ri-Gui Zhou and Da-Qian Liu. “Quantum image edge extraction based on improved Sobel operator”. In: *International Journal of Theoretical Physics* 58 (2019), pp. 2969–2985.

.1 Appendix A: Verification of Effectiveness

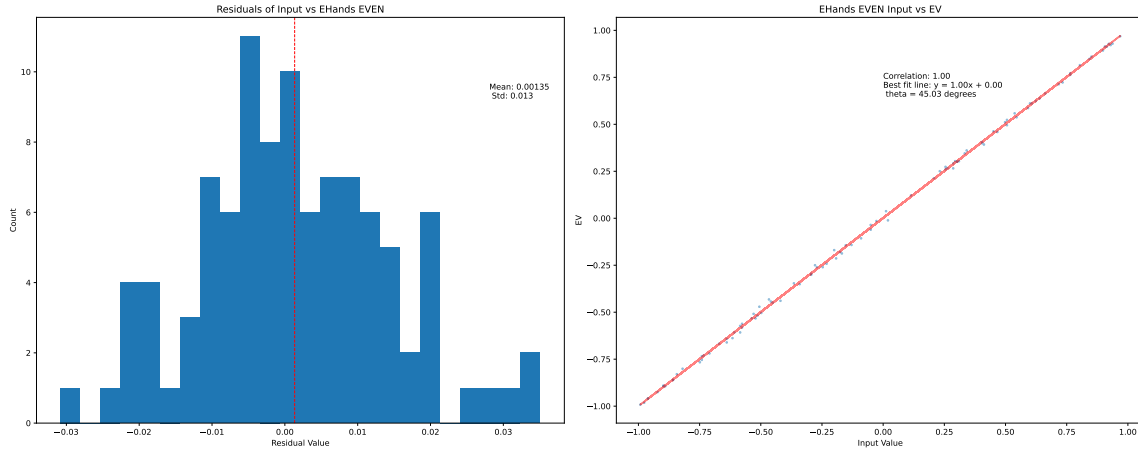


Figure .1: Accuracy Verification for EHands Encoding with 100 randomly generated samples

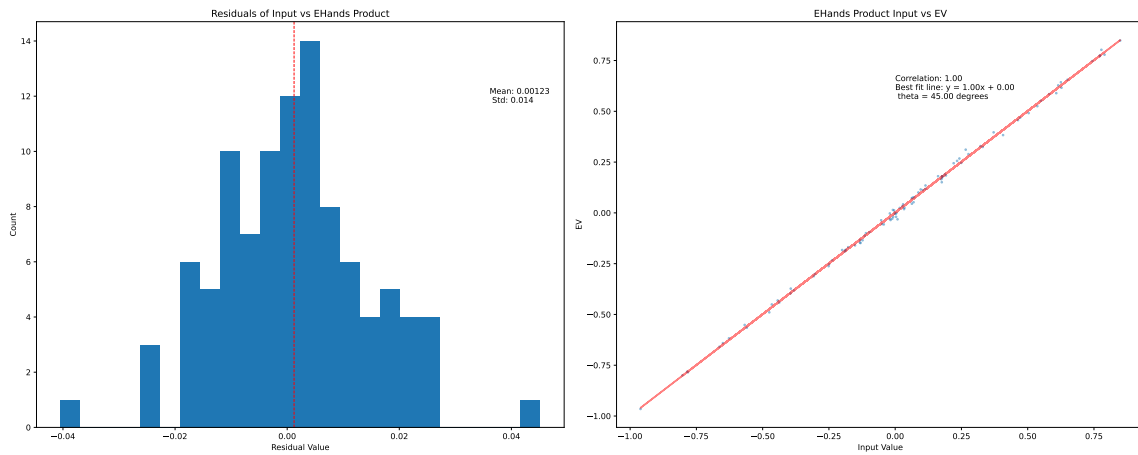


Figure .2: Verification of Effectiveness for Ehands Multiplication with 100 randomly generated samples

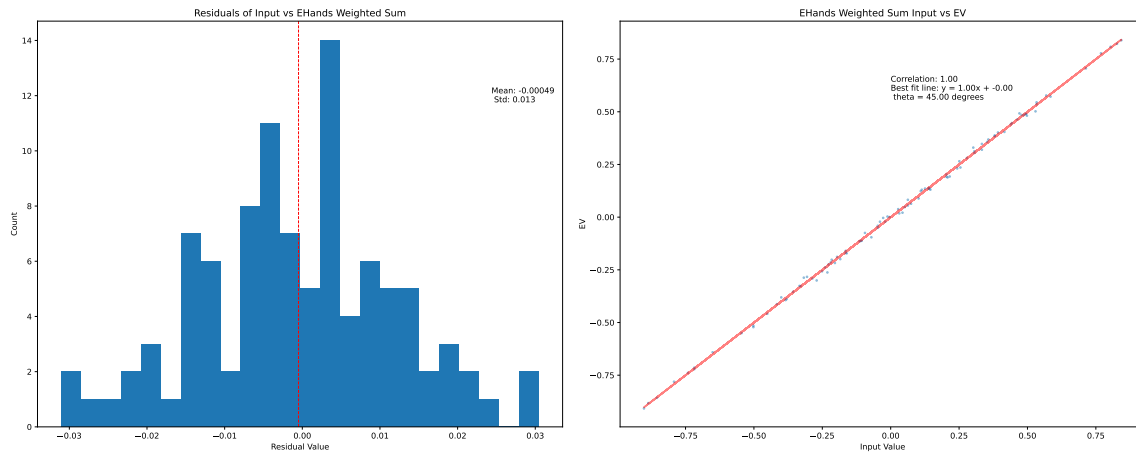


Figure .3: Verification of Effectiveness for EHands Weighted Sum with 100 randomly generated samples

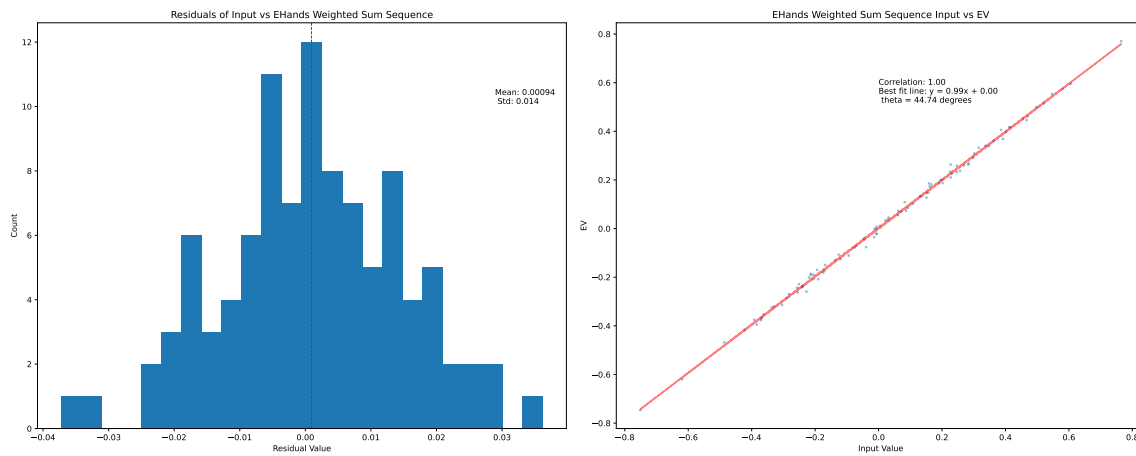


Figure .4: Verification of Effectiveness for the new parity flip block using a sequence of EHands weighted summations with 100 randomly generated samples

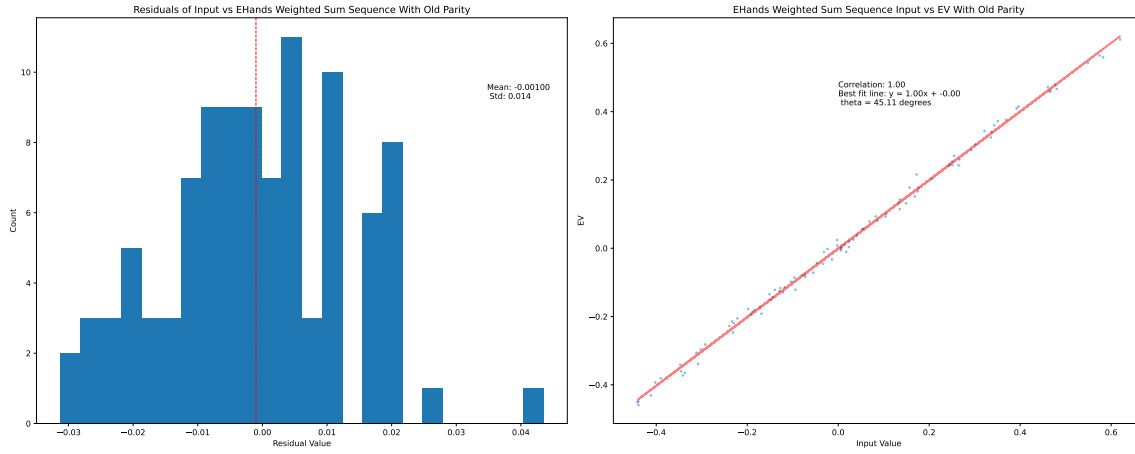


Figure .5: Verification of Effectiveness for the old parity flip block using a sequence of EHands weighted summations with 100 randomly generated samples

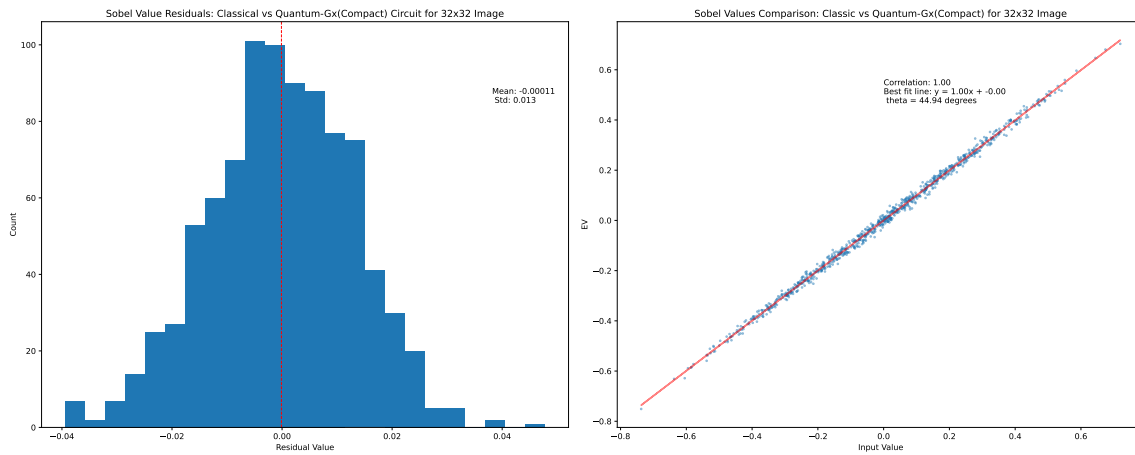


Figure .6: Verification of Effectiveness of the G_x compact circuit with a 32×32 image

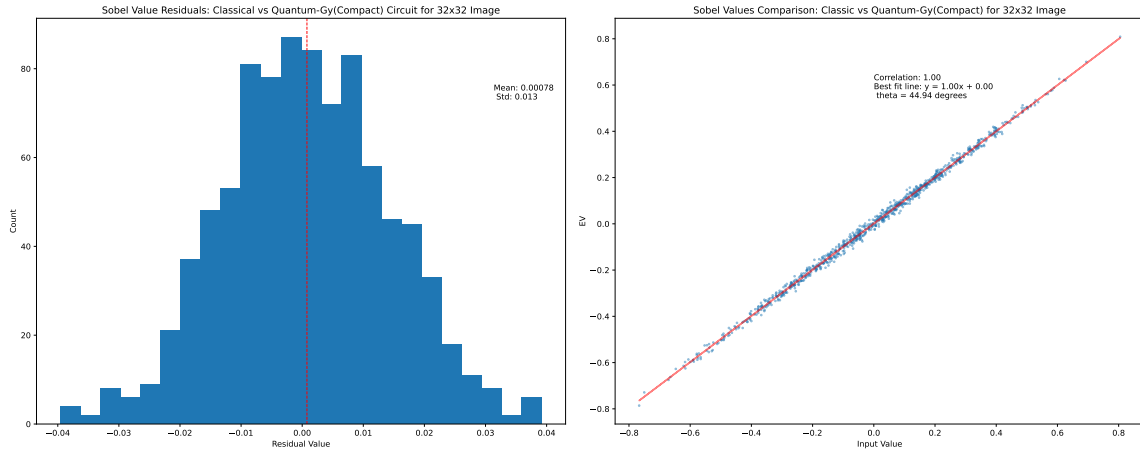


Figure .7: Verification of Effectiveness of the G_y compact circuit with a 32×32 image

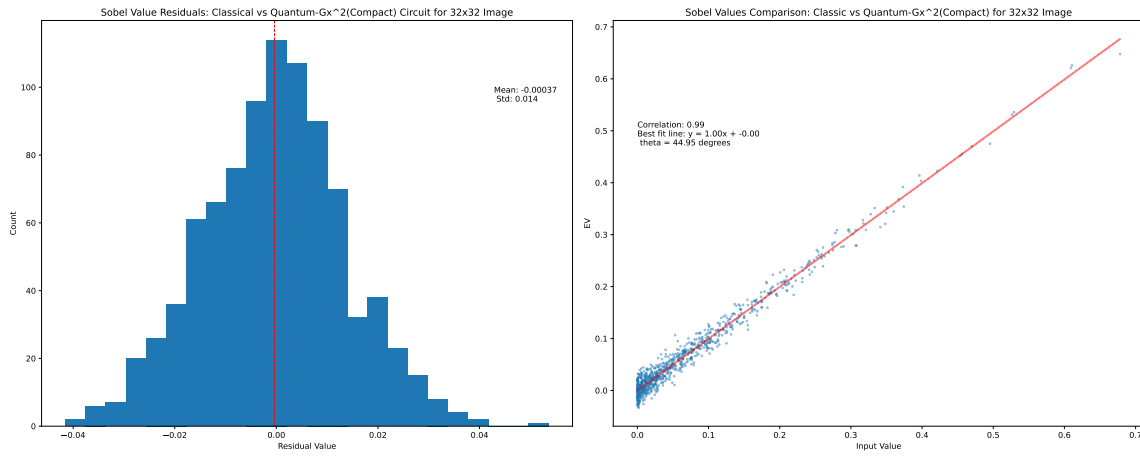


Figure .8: Verification of Effectiveness of the G_x^2 compact circuit with a 32×32 image

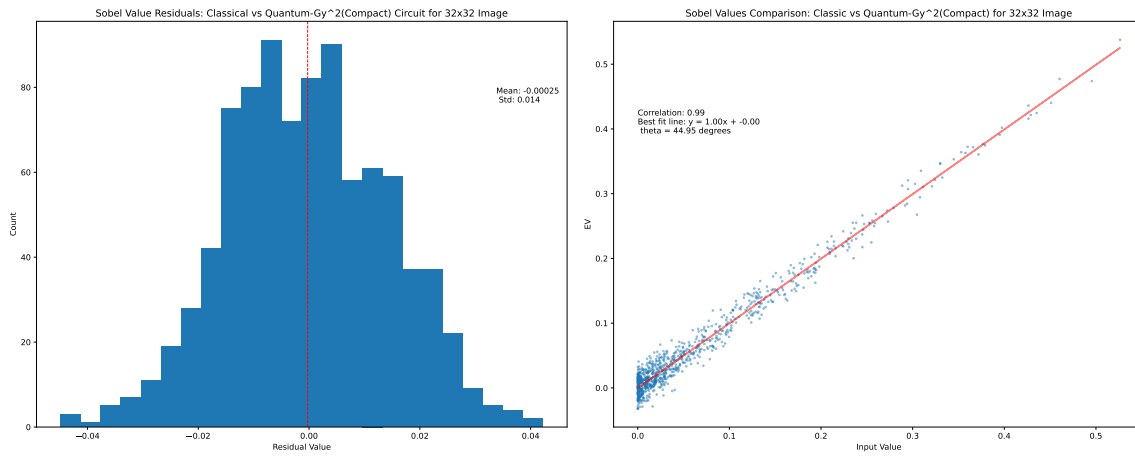


Figure .9: Verification of Effectiveness of the G_y^2 compact circuit with a 32×32 image